Multi-threaded File Download Client/Server

คู่มืออธิบายโค้ด

Workflow · Syntax ของ Java ที่ใช้ · อธิบายทีละไฟล์ · ผลการตรวจสอบโค้ด

เอกสารนี้ใช้ทำอะไร

- เข้าใจว่าโปรแกรมทำงานตั้งแต่พิมพ์คำสั่งจนได้ไฟล์ครบอย่างไร (Workflow)
- อ่านโค้ด Java ของโปรเจกต์ได้ แม้เพิ่งเริ่มเขียน (อธิบาย syntax จากโค้ดจริงของโปรเจกต์)
- รู้ว่าแต่ละไฟล์ (13 ไฟล์) ทำหน้าที่อะไร และมีจุดไหนต้องระวัง
- ดูผลการตรวจสอบโค้ดเทียบกับโจทย์ บั๊กที่พบและแก้แล้ว และข้อจำกัดที่ยังเหลือ
- เตรียมตอบคำถามเวลานำเสนอ

ตัวอย่างโค้ดทุกชิ้นในเอกสารนี้ตัดมาจากไฟล์ .java ที่ส่งมอบจริงโดยอัตโนมัติ พร้อมเลขบรรทัด จึงตรงกับโค้ดเสมอ · ตรวจสอบเมื่อ 13 ตุลาคม 2569 (2026) · TestRunner ผ่าน 84/84 ข้อ

สารบัญ

ส่วนที่ 1 ภาพรวมของโปรแกรม 3

ส่วนที่ 2 Workflow: โปรแกรมทำงานอย่างไร 5

ส่วนที่ 3 Syntax ของ Java ที่ใช้ในโปรเจกต์ 9

ส่วนที่ 4 อธิบายทีละไฟล์ 16

Protocol.java · ค่าคงที่ของ protocol + อ่าน/ส่งข้อความ 1 บรรทัด 16

FileUtil.java · คำนวณ SHA-256 ของไฟล์ 17

Connection.java · TCP connection หนึ่งเส้น 18

Server.java · main ของ server: เปิดพอร์ต รับ connection 19

ClientHandler.java · คุยกับ client หนึ่งราย (1 thread ต่อ 1 connection) 20

FileSender.java · ส่งข้อมูลช่วงที่ขอ: แบบ io และแบบ nio 22

Client.java · main ของ client: อ่านคำสั่งจาก command line 23

Downloader.java · งานดาวน์โหลดทั้งหมด: แบ่งช่วง สั่ง worker ตรวจไฟล์ 24

DownloadResult.java · กล่องเก็บผลของการดาวน์โหลดหนึ่งครั้ง 26

RangeWorker.java · worker 1 ตัว: connection ของตัวเอง ช่วงของตัวเอง 26

Benchmark.java · รันการทดลองและเก็บผลเป็นตาราง/CSV 28

MakeTestFile.java · สร้างไฟล์สุ่มขนาดใหญ่สำหรับทดลอง 30

TestRunner.java · test case อัตโนมัติ 84 ข้อ 30

ส่วนที่ 5 ตรวจโค้ดเทียบกับโจทย์ 32

ส่วนที่ 6 บั๊กที่พบ ข้อจำกัด และผลทดสอบ 34

ส่วนที่ 7 คำถามที่อาจถูกถามตอนนำเสนอ 36

ภาคผนวก คำสั่ง รหัส และศัพท์ 38

ส่วนที่ 1 ภาพรวมของโปรแกรม

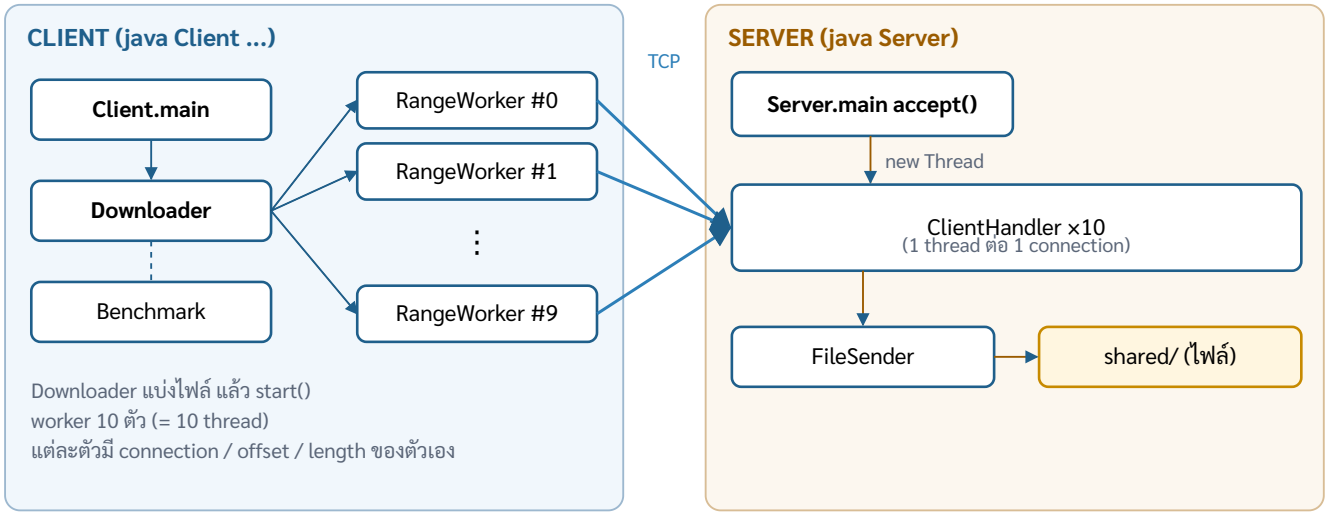
1.1 โปรแกรมนี้ทำอะไร

โปรแกรมนี้คือระบบ **Client/Server** ที่คุยกันผ่าน **TCP** เพื่อดาวน์โหลดไฟล์ โดย Client จะ**แบ่งไฟล์เป็น 10 ช่วง** (byte range) แล้วใช้ **worker 10 ตัว (10 thread)** ดาวน์โหลดพร้อมกัน แต่ละตัวมี connection ของตัวเอง ขอข้อมูลเฉพาะช่วงของตัวเอง แล้วเขียนลงไฟล์ปลายทางตรงตำแหน่งเดียวกับในไฟล์ต้นฉบับ เมื่อครบทุกช่วง Client ตรวจ**ขนาด**และ **SHA-256** ว่าไฟล์ไม่เสียหาย

โปรแกรมมีวิธีคัดลอกข้อมูล 2 แบบ เพื่อนำไปเปรียบเทียบเวลาและ throughput ตามโจทย์:

	Traditional I/O (โหมด `io`)	**NIO native transfer (โหมด `nio`)**
ฝั่ง Server	อ่านไฟล์ใส่ `byte[]` ด้วย `RandomAccessFile.read` แล้ว `OutputStream.write` ลง socket	`FileChannel.transferTo(...)` ให้ระบบปฏิบัติการส่งจากไฟล์ไป socket เอง
ฝั่ง Client	อ่าน socket ใส่ `byte[]` แล้ว `RandomAccessFile.write` ที่ตำแหน่ง offset	`FileChannel.transferFrom(socketChannel, position, count)`

1.2 แผนภาพโครงสร้าง



รูปที่ 1 โครงสร้างโดยรวม: worker แต่ละตัวมี connection ของตัวเอง และ server สร้าง thread ให้ทุก connection

1.3 ไฟล์ทั้งหมดในโปรเจกต์ (13 ไฟล์ .java)

#	ไฟล์	ฝั่ง	หน้าที่	บรรทัด
1	`Protocol.java`	ใช้ร่วมกัน	ค่าคงที่ของ protocol + อ่าน/ส่งข้อความ 1 บรรทัด	59
2	`FileUtil.java`	ใช้ร่วมกัน	คำนวณ SHA-256 ของไฟล์	38

3	`Connection.java`	Client	TCP connection หนึ่งเส้น	57
4	`Server.java`	Server	main ของ server: เปิดพอร์ต รับ connection	79
5	`ClientHandler.java`	Server	คุยกับ client หนึ่งราย (1 thread ต่อ 1 connection)	209
6	`FileSender.java`	Server	ส่งข้อมูลช่วงที่ขอ: แบบ io และแบบ nio	57
7	`Client.java`	Client	main ของ client: อ่านคำสั่งจาก command line	153
8	`Downloader.java`	Client	งานดาวน์โหลดทั้งหมด: แบ่งช่วง สั่ง worker ตรวจไฟล์	93
9	`DownloadResult.java`	Client	กล่องเก็บผลของการดาวน์โหลดหนึ่งครั้ง	10
10	`RangeWorker.java`	Client	worker 1 ตัว: connection ของตัวเอง ช่วงของตัวเอง	122
11	`Benchmark.java`	Client	รันการทดลองและเก็บผลเป็นตาราง/CSV	115
12	`MakeTestFile.java`	เครื่องมือ	สร้างไฟล์สุ่มขนาดใหญ่สำหรับทดลอง	44
13	`TestRunner.java`	เครื่องมือ	test case อัตโนมัติ 84 ข้อ	740

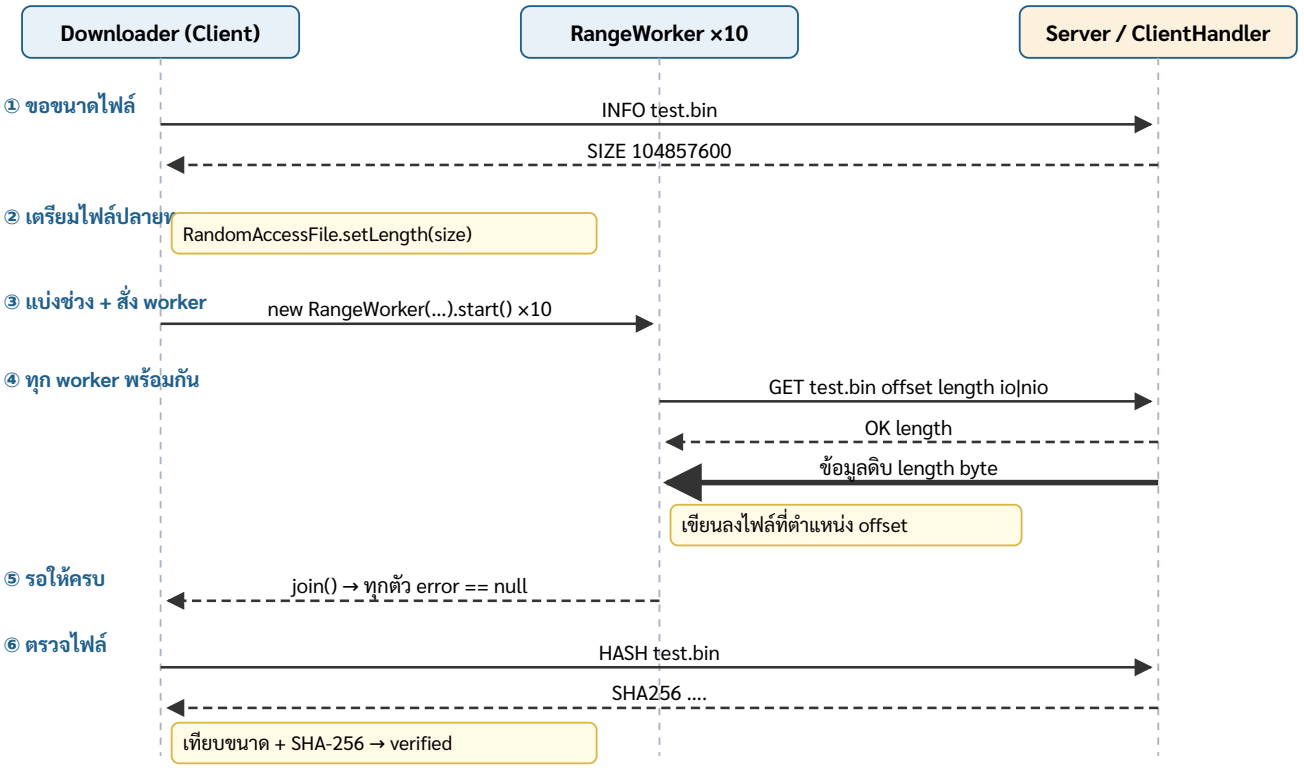
นอกจากนี้มี `README_THAI.txt`, `README_ENG.txt` (อธิบายวิธีทำและเหตุผล), `HOW_TO_RUN.txt` (คู่มือรันทีละขั้น), `testcase.txt` (test case 10 ข้อพร้อมผลจริง) และโฟลเดอร์ `shared/` (ไฟล์ที่ server แชร์)

1.4 คำสั่งที่ใช้บ่อย

```
javac *.java                                        # compile (ทำใหม่ทุกครั้งที่แก้โค้ด)
java TestRunner                                     # test อัตโนมัติ 84 ข้อ
java MakeTestFile shared/test.bin 100               # สร้างไฟล์สุ่ม 100 MB
java Server                                         # เปิด server (เทอร์มินัล 1)
java Client localhost 5000 list                     # ดูรายชื่อไฟล์ (เทอร์มินัล 2)
java Client localhost 5000 info test.bin            # ดูขนาดไฟล์
java Client localhost 5000 download test.bin out.bin 10 nio   # 10 workers แบบ nio
java Client localhost 5000 benchmark test.bin 3  # ทดลองทุกกรณี 3 รอบ + เก็บ CSV
```

ส่วนที่ 2 Workflow: โปรแกรมทำงานอย่างไร

2.1 ลำดับการดาวน์โหลด (ตั้งแต่สั่งคำสั่งจนไฟล์เสร็จ)



รูปที่ 2 ลำดับข้อความระหว่าง Downloader, RangeWorker และ Server (เส้นหนา = ข้อมูลไฟล์, เส้นประ = คำตอบ)

1. **Client.main** อ่านคำสั่ง `download test.bin out.bin 10 nio` แล้วเรียก `Downloader.download(...)`
2. Downloader ส่ง `INFO` เพื่อรู้**ขนาดไฟล์** (เช่น 104,857,600 byte)
3. สร้างไฟล์ปลายทางและ**ตั้งขนาดให้เต็ม**ด้วย `setLength(size)` ตั้งแต่ต้น เพื่อให้ทุก worker เขียนลงตำแหน่งของตัวเองได้
4. คำนวณช่วงของแต่ละ worker (`chunk = size / workers`, ตัวสุดท้ายรับส่วนที่เหลือ) แล้ว `new RangeWorker(...).start()` 10 ตัว
5. worker แต่ละตัว**เปิด connection ใหม่ของตัวเอง** ส่ง `GET ไฟล์ offset length โหมด` รับ `OK length` แล้วรับข้อมูลดิบ และเขียนลงไฟล์ที่ตำแหน่ง offset
6. Downloader `join()` รอทุก worker ถ้าตัวไหนมี `error` จะลบไฟล์ที่ยังไม่ครบทิ้งและรายงาน
7. ตรวจ**ขนาด**และ **SHA-256** ของไฟล์ที่ได้เทียบกับ `HASH` ที่ server คำนวณ แล้วพิมพ์ผล (`OK` หรือ `MISMATCH`)

> **เวลาที่วัด (time) หมายถึงช่วงไหน**
> วัดตั้งแต่เริ่มสร้าง worker จนทุก worker เสร็จ **ไม่รวม**การสร้างไฟล์ปลายทางและการตรวจ SHA-256 (ดู `Downloader.java` ขั้นที่ 3–4 และ 6)

2.2 Protocol: ภาษาที่ Client กับ Server คุยกัน

ทุกคำขอเป็น**ข้อความ 1 บรรทัด** ลงท้ายด้วย `\n` คำตอบเป็นข้อความ 1 บรรทัดเช่นกัน ยกเว้น `GET` ที่ตอบบรรทัด `OK` แล้วตามด้วย**ข้อมูลดิบ (raw bytes) จำนวนเท่าที่ประกาศไว้พอดี**

คำขอ	คำตอบ	ใช้ทำอะไร
`LIST`	`FILE <ชื่อ> <ขนาด>` (หนึ่งบรรทัดต่อไฟล์) แล้วจบด้วย `END`	ขอรายชื่อไฟล์
`INFO <ไฟล์>`	`SIZE <bytes>`	ขอขนาดก่อนเริ่มโหลด
`HASH <ไฟล์>`	`SHA256 <hex 64 ตัว>`	ขอลายนิ้วมือของไฟล์ไว้ตรวจ (คำสั่งที่เพิ่มเอง)
`GET <ไฟล์> <offset> <length> [io|nio]`	`OK <length>` + ข้อมูลดิบ `length` byte	ขอเฉพาะช่วงของไฟล์ (ไม่ใส่โหมด = io)
(มีปัญหา)	`ERROR <code> <ข้อความ>`	400 = คำขอผิด · 404 = ไม่พบไฟล์ · 416 = ช่วงเกินขนาดไฟล์

ตัวอย่างบทสนทนาจริง (→ client ส่ง, ← server ตอบ)

```
→ LIST
← FILE hello.txt 44
← FILE sample_odd.bin 1000003
← END
→ INFO hello.txt
← SIZE 44
→ HASH hello.txt
← SHA256 3ccca89ac63e1f0294ed14490efdaeeba4ade11bef4882afc8069b9c8e91e36a
→ GET hello.txt 6 4 io
← OK 4
← from                      (ข้อมูลดิบ 4 byte ต่อจากบรรทัด OK)
→ GET nope.bin 0 10
← ERROR 404 File not found
→ GET hello.txt 40 100
← ERROR 416 Range out of bounds (file size is 44)
```

> **ทำไมต้องบอกจำนวน byte ล่วงหน้า (OK length)**
> ข้อมูลไฟล์เป็น byte ธรรมดาที่ไม่มีเครื่องหมายสิ้นสุด ฝั่งรับจึงต้องรู้ว่าต้องอ่านกี่ byte จึงจะครบ และต้องอ่าน header เป็นข้อความโดย**ไม่อ่านเกิน**บรรทัดนั้น ไม่เช่นนั้นจะกลืน byte แรก ๆ ของไฟล์ไปด้วย (ดู `Protocol.readLine`)

2.3 การแบ่งช่วงข้อมูลให้ 10 worker

ตัวอย่าง A: ไฟล์ 1,000,003 byte (หารด้วย 10 ไม่ลงตัว) → chunk = 100,000

worker 0	worker 1	worker 2	worker 3	worker 4	worker 5	worker 6	worker 7	worker 8	worker 9
offset 0	offset 100,000	offset 200,000	offset 300,000	offset 400,000	offset 500,000	offset 600,000	offset 700,000	offset 800,000	offset 900,000
length 100,000	length 100,000	length 100,000	length 100,000	length 100,000	length 100,000	length 100,000	length 100,000	length 100,000	length **100,003**

worker ตัวสุดท้ายได้ length = size − offset = 1,000,003 − 900,000 = 100,003 (รับส่วนที่เหลือ)

ตัวอย่าง B: ไฟล์ 44 byte (hello.txt) → chunk = 44 ÷ 10 = 4

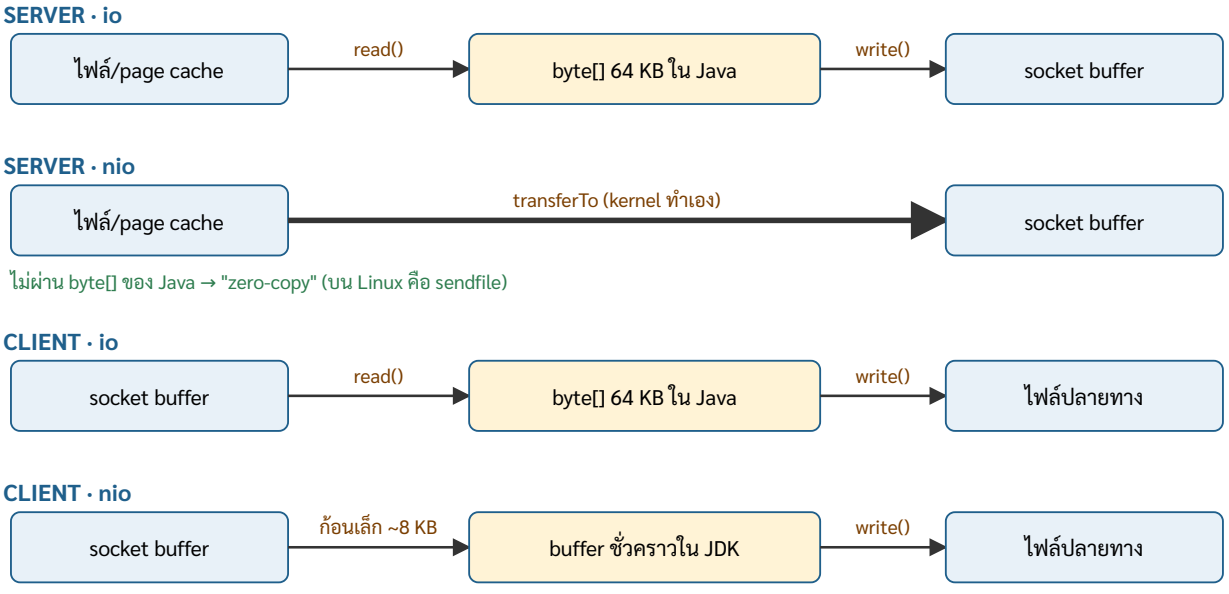
off 0 · len 4	off 4 · len 4	off 8 · len 4	off 12 · len 4	off 16 · len 4	off 20 · len 4	off 24 · len 4	off 28 · len 4	off 32 · len 4	off 36 · len 8

44 − 36 = 8 : worker ตัวสุดท้ายได้ 8 byte (ตรงกับผลจริงใน testcase.txt ข้อ TC08)

รูปที่ 3 สูตร: chunk = size ÷ workers (หารปัดเศษทิ้ง) · offset ของ worker i = i × chunk · ตัวสุดท้ายได้ length = size − offset

- ช่วงของทุก worker **ต่อกันพอดี ไม่ซ้อน ไม่เว้น** เพราะ offset ของตัวถัดไป = offset + length ของตัวก่อนหน้า
- ไฟล์เล็กกว่า 10 byte (เช่น 9 byte): chunk = 0 ดังนั้น worker 0–8 ได้ length 0 และตัวสุดท้ายได้ทั้งไฟล์ — โปรแกรมยังทำงานถูกต้อง (มีใน TestRunner)
- ไฟล์ขนาด 0 byte: ทุก worker ได้ length 0 และไฟล์ที่ได้เป็นไฟล์ว่าง (มีใน TestRunner)

2.4 เส้นทางของข้อมูล: io เทียบกับ nio



รูปที่ 4 ข้อมูลเดินทางผ่านอะไรบ้างในแต่ละโหมด (กล่องสีเหลือง = พื้นที่ของโปรแกรม Java)

- **โหมด io:** ข้อมูลถูกคัดลอกเข้า-ออก `byte[]` ของโปรแกรมทุกรอบ (ระบบปฏิบัติการ → Java → ระบบปฏิบัติการ)
- **โหมด nio ฝั่ง server:** `transferTo` ให้ระบบปฏิบัติการส่งข้อมูลจากไฟล์ไป socket โดยตรง ไม่ผ่าน `byte[]` (บน Linux ใช้ `sendfile`) ลดการคัดลอกและการสลับระหว่างโหมดผู้ใช้/เคอร์เนล

- **โหมด nio ฝั่ง client:** `transferFrom` จาก socket ไม่ใช่ zero-copy จริง JDK ยังอ่านเป็นก้อนเล็ก (ประมาณ 8 KB) แล้วเขียนลงไฟล์ จึงไม่แปลกที่ nio ฝั่ง client จะไม่เร็วกว่า io

2.5 Thread ในโปรแกรมนี้

ที่ไหน	thread อะไร	จำนวน	หน้าที่
Server	main thread	1	วน `accept()` รอ client
Server	`ClientHandler` (สร้างด้วย `new Thread`)	1 ต่อ 1 connection	อ่านคำสั่งและตอบ client รายนั้น
Client	main thread	1	สร้าง worker, `join()` รอ, ตรวจไฟล์
Client	`RangeWorker` (`extends Thread`)	10 (ตั้งได้ 1–100)	โหลดช่วงของตัวเอง

> **ทำไม worker ไม่ต้องใช้ lock**
> worker แต่ละตัวมี connection และ file handle ของตัวเอง และเขียนเฉพาะช่วง `[offset, offset+length)` ของตัวเอง ซึ่งไม่ซ้อนกับใคร จึงไม่มีข้อมูลที่ใช้ร่วมกัน (ไม่เกิด race condition) ตัวแปร `error` ของ worker ถูกอ่านหลัง `join()` เท่านั้น และ `join()` รับประกันว่าค่าที่ worker เขียนไว้ main thread มองเห็นแน่นอน

ส่วนที่ 3 Syntax ของ Java ที่ใช้ในโปรเจกต์

ส่วนนี้อธิบายโครงสร้างภาษา Java ที่พบในโค้ด **โดยยกตัวอย่างจากโค้ดจริงของโปรเจกต์** อ่านจบแล้วจะอ่านทุกไฟล์ในส่วนที่ 4 รู้เรื่อง

3.1 class และ main

โค้ด Java ทุกบรรทัดอยู่ใน `class` ชื่อ class ต้องตรงกับชื่อไฟล์ (`Server` → `Server.java`) โปรแกรมเริ่มทำงานที่เมธอด `main` ซึ่งต้องเขียนเป๊ะ ๆ แบบนี้ ส่วน `String[] args` คือคำที่พิมพ์ต่อท้ายคำสั่ง เช่น `java Server 6000 myfolder` ให้ `args[0]` = "6000" และ `args[1]` = "myfolder"

โค้ดจริงจากไฟล์ `Server.java` · บรรทัด 14–16

```
public class Server {

    public static void main(String[] args) throws IOException {
```

- `public` = ไฟล์อื่นเรียกใช้ได้ · `static` = เรียกได้โดยไม่ต้องสร้าง object · `void` = ไม่คืนค่า
- `throws IOException` = เมธอดนี้อาจโยน error ประเภท IOException ออกไปให้ผู้เรียกจัดการ

3.2 ชนิดข้อมูลและตัวแปร

โค้ดจริงจากไฟล์ `DownloadResult.java` · บรรทัด 5–10

```
public class DownloadResult {
    public long sizeBytes;       // size of the file
    public double seconds;       // time used by the workers
    public double mbPerSec;      // throughput in MB/s
    public boolean verified;     // true if size and SHA-256 are the same as the original
}
```

ชนิด	เก็บอะไร	ตัวอย่างในโปรเจกต์
`int`	จำนวนเต็ม 32 บิต (ถึงประมาณ ±2,147 ล้าน)	`port`, `workers`, ตัวนับ `i`
`long`	จำนวนเต็ม 64 บิต	`size`, `offset`, `length` — ไฟล์ใหญ่กว่า 2 GB ต้องใช้ long
`double`	ทศนิยม	`seconds`, `mbPerSec`
`boolean`	true / false	`verified`, `useNio`
`String`	ข้อความ	ชื่อไฟล์, บรรทัด protocol
`byte[]`	อาร์เรย์ของ byte (ที่พักข้อมูลชั่วคราว)	`new byte[Protocol.BUFFER_SIZE]` ขนาด 64 KB

ทำไมต้องใช้ long: ตัวอย่างบั๊กที่เกิดได้กับ int
ไฟล์ 2.5 GB มีขนาด 2,621,440,000 byte ซึ่งเกินค่าสูงสุดของ `int` (2,147,483,647) ถ้าเก็บใน `int` ตัวเลขจะ**ล้นกลายเป็นค่าติดลบ** เราทดลองดาวน์โหลดไฟล์ 2.5 GB ด้วย 10 workers ทั้ง io และ nio แล้ว SHA-256 ตรงกับต้นฉบับ แสดงว่า offset ที่เกิน 231 ทำงานถูก

โค้ดจริงจากไฟล์ `Downloader.java` · บรรทัด 49–58

```
long chunk = size / workers;
RangeWorker[] list = new RangeWorker[workers];

long startTime = System.nanoTime();
for (int i = 0; i < workers; i++) {
    long offset = i * chunk;
    long length = chunk;
    if (i == workers - 1) {
        length = size - offset;
    }
```

สังเกต `long offset = i * chunk;` : `i` เป็น int แต่ `chunk` เป็น long ผลคูณจึงถูกคำนวณเป็น long ไม่ล้น

3.3 if / while / for และการแปลงชนิด (cast)

โค้ดจริงจากไฟล์ `FileSender.java` · บรรทัด 23–30

```
while (remaining > 0) {
    int toRead = (int) Math.min(buffer.length, remaining);
    int n = raf.read(buffer, 0, toRead);          // n = how many bytes we really got
    if (n == -1) {
        throw new IOException("File ended too early");
    }
    out.write(buffer, 0, n);
    remaining = remaining - n;
```

- `while (เงื่อนไข) { ... }` วนซ้ำตราบที่เงื่อนไขเป็นจริง — โปรเจกต์นี้ใช้ while เพื่อ**วนจนส่ง/รับครบ** เพราะการอ่าน/เขียนครั้งเดียวอาจได้ไม่ครบ
- `(int) Math.min(buffer.length, remaining)` : `Math.min` เลือกค่าที่น้อยกว่า ผลเป็น `long` เพราะ remaining เป็น long จึงต้อง **cast** ด้วย `(int)` ก่อนส่งให้ `read` — ปลอดภัย เพราะค่าไม่เกินขนาด buffer (65,536)
- `n == -1` คือสัญญาณว่า**ไฟล์/connection จบแล้ว** (ไม่ใช่จำนวน byte)
- `remaining = remaining - n;` ลดจำนวน byte ที่ยังค้างด้วยจำนวนที่ทำได้จริงในรอบนั้น (ไม่ใช่ขนาด buffer)

3.4 try / catch / finally และ exception

Exception คือ "ข้อผิดพลาดที่ถูกโยนออกมา" เช่น connection หลุด โค้ดที่เสี่ยงใส่ใน `try` ถ้าเกิดข้อผิดพลาดจะกระโดดไป `catch` ส่วน `finally` ทำเสมอไม่ว่าจะสำเร็จหรือพัง (ใช้ปิด connection)

โค้ดจริงจากไฟล์ `ClientHandler.java` · บรรทัด 34–59

```
public void run() {
    try {
        in = channel.socket().getInputStream();
        out = channel.socket().getOutputStream();
        channel.socket().setSoTimeout(idleTimeoutMs);    // only affects READING commands

        while (true) {
            String line = Protocol.readLine(in);
            if (line == null) {
                break;                       // the client closed the connection
            }
            handleCommand(line);
        }
    } catch (SocketTimeoutException e) {
        System.out.println("[server] closing a connection that was idle for too long");
    } catch (IOException e) {
        // normal when a client disconnects in the middle of a transfer
        System.out.println("[server] connection ended: " + e.getMessage());
    } finally {
        try {
            channel.close();                 // always close the connection at the end
        } catch (IOException e) {
            // nothing more we can do
        }
    }
}
```

- `catch (SocketTimeoutException e)` ต้องเขียน**ก่อน** `catch (IOException e)` เพราะ SocketTimeoutException เป็นลูกของ IOException ถ้าสลับลำดับ Java จะ compile ไม่ผ่าน (catch แรกครอบอยู่แล้ว)
- `throw new IOException("...")` คือการโยน error เอง เช่น เมื่อ server ตอบ `ERROR` (ดู `Connection.readReply`)

3.5 try-with-resources และ AutoCloseable

ทรัพยากรอย่าง connection หรือไฟล์ต้อง "ปิด" เสมอ ถ้าใส่ไว้ในวงเล็บของ `try (...)` Java จะเรียก `close()` ให้อัตโนมัติเมื่อจบบล็อก แม้เกิด exception กลางทาง คลาสที่ใช้แบบนี้ได้ต้อง `implements AutoCloseable` (เช่น `Connection`)

โค้ดจริงจากไฟล์ `Downloader.java` · บรรทัด 15–22

```
public static long getFileSize(String host, int port, String filename) throws IOException {
    try (Connection c = new Connection(host, port)) {
        c.sendLine("INFO " + filename);
        String reply = c.readReply();                  // "SIZE 104857600"
        String[] words = reply.split(" ");
        return Long.parseLong(words[1]);
    }
}
```

โค้ดจริงจากไฟล์ `Connection.java` · บรรทัด 14–14

```
public class Connection implements AutoCloseable {
```

โค้ดจริงจากไฟล์ `Connection.java` · บรรทัด 54–56

```
public void close() throws IOException {
    channel.close();
}
```

3.6 Thread: extends Thread, implements Runnable, start() และ join()

- `class X extends Thread` หรือ `class X implements Runnable` แล้วเขียนงานไว้ในเมธอด `run()`
- เรียก **`start()`** = สร้าง thread ใหม่แล้วให้มันรัน `run()` **ขนานไปกับ** thread ปัจจุบัน (ถ้าเรียก `run()` ตรง ๆ จะรันใน thread เดิม ไม่ขนาน!)
- เรียก **`join()`** = รอจนกว่า thread นั้นจะทำงานเสร็จ

โค้ดจริงจากไฟล์ `RangeWorker.java` · บรรทัด 14–14

```
public class RangeWorker extends Thread {
```

โค้ดจริงจากไฟล์ `Downloader.java` · บรรทัด 59–61

```
list[i] = new RangeWorker(i, host, port, filename, outFile,
        offset, length, useNio, verbose);
list[i].start();                              // the worker starts running in parallel
```

โค้ดจริงจากไฟล์ `Downloader.java` · บรรทัด 64–67

```
// 4) wait until every worker has finished
for (int i = 0; i < workers; i++) {
    list[i].join();
}
```

โค้ดจริงจากไฟล์ `Server.java` · บรรทัด 66–68

```
ClientHandler handler = new ClientHandler(client, folder);
Thread thread = new Thread(handler);
thread.start();                                  // handler.run() now runs in parallel
```

โค้ดจริงจากไฟล์ `ClientHandler.java` · บรรทัด 16–16

```
public class ClientHandler implements Runnable {
```

3.7 static / final / public / private

โค้ดจริงจากไฟล์ `Protocol.java` · บรรทัด 22–25

```
public static final int DEFAULT_PORT = 5000;
public static final int BUFFER_SIZE = 64 * 1024;     // 64 KB
public static final String MODE_IO = "io";           // Traditional I/O
public static final String MODE_NIO = "nio";         // NIO transferTo / transferFrom
```

- `static final` = ค่าคงที่ของ class เปลี่ยนไม่ได้ ใช้ได้ทุกที่ด้วย `Protocol.BUFFER_SIZE`
- `private` = ใช้ได้เฉพาะภายใน class ตัวเอง (เช่นเมธอด `handleGet` ใน ClientHandler) ส่วน `public` = เรียกจากไฟล์อื่นได้
- `64 * 1024` = 65,536 byte = 64 KB (ขนาดที่พักข้อมูลต่อรอบ)

3.8 เมธอดของ String ที่ใช้

โค้ดจริงจากไฟล์ `ClientHandler.java` · บรรทัด 63–67

```
String[] parts = line.split("\\s+");     // split on spaces, e.g. "GET a.bin 0 10"
// Locale.ROOT = always the English rules. (With the Turkish rules "info".toUpperCase()
// would give "İNFO", and the command would not be recognised.)
String command = parts[0].toUpperCase(Locale.ROOT);
```

เมธอด	ทำอะไร	ตัวอย่าง
`split("\\s+")`	ตัดข้อความที่ช่องว่าง (regex `\s+` = ช่องว่างหนึ่งตัวขึ้นไป)	"GET a.bin 0 10" → ["GET","a.bin","0","10"]
`equals` / `equalsIgnoreCase`	เทียบข้อความ (ห้ามใช้ `==` กับ String)	`command.equals("LIST")`
`startsWith`	ขึ้นต้นด้วยข้อความที่กำหนดไหม	`reply.startsWith("ERROR")`
`trim()`	ตัดช่องว่าง/ขึ้นบรรทัดหน้า-หลังออก	ตัด `\r` ท้ายบรรทัดที่ Windows ส่งมา
`Long.parseLong` / `Integer.parseInt`	แปลงข้อความเป็นตัวเลข (ถ้าไม่ใช่ตัวเลขจะโยน NumberFormatException)	`Long.parseLong(parts[2])`

3.9 printf / String.format และ Locale.ROOT

โค้ดจริงจากไฟล์ `RangeWorker.java` · บรรทัด 76–77

```
System.out.printf(Locale.ROOT, "  worker %2d  offset=%-12d length=%-12d %.0f ms%n",
        workerId, offset, length, (System.nanoTime() - startTime) / 1e6);
```

รูปแบบ	ความหมาย
`%d`, `%2d`, `%-12d`	จำนวนเต็ม · กว้างอย่างน้อย 2 ช่อง · ชิดซ้ายกว้าง 12 ช่อง
`%.0f`, `%.3f`	ทศนิยม 0 ตำแหน่ง · 3 ตำแหน่ง
`%s`, `%n`	ข้อความ · ขึ้นบรรทัดใหม่
`%02x`	เลขฐาน 16 อย่างน้อย 2 หลัก เติม 0 ข้างหน้า (ใช้แปลง byte เป็น hex ใน SHA-256)

Locale.ROOT: บั๊กจริงที่เคยพบ

`String.format("%.3f", 0.006)` เขียนตาม**ภาษาของเครื่อง** บนเครื่องภาษาเยอรมันได้ `0,006` (ใช้จุลภาค) ทำให้ไฟล์ CSV ที่คั่นด้วยจุลภาค**คอลัมน์พัง** (แถวหนึ่งมี 9 ช่องแทนที่จะเป็น 6) เราจึงใส่ `Locale.ROOT` ให้ตัวเลขใช้จุดทศนิยมเสมอ และเพิ่ม test ที่จำลองเครื่องภาษาเยอรมัน (test นี้ FAIL กับโค้ดก่อนแก้ จึงพิสูจน์ได้ว่าจับบั๊กจริง) ในทำนองเดียวกัน `toUpperCase(Locale.ROOT)` กัน "info" กลายเป็น "İNFO" บนเครื่องภาษาตุรกี

3.10 Stream: อ่าน/เขียน byte

โค้ดจริงจากไฟล์ `RangeWorker.java` · บรรทัด 83–99

```
private void receiveWithStreams(Connection c) throws IOException {
    try (RandomAccessFile raf = new RandomAccessFile(outFile, "rw")) {
        raf.seek(offset);                                   // jump to our part of the file
        byte[] buffer = new byte[Protocol.BUFFER_SIZE];
        long remaining = length;                            // bytes still missing

        while (remaining > 0) {
            int toRead = (int) Math.min(buffer.length, remaining);
            int n = c.in.read(buffer, 0, toRead);
            if (n == -1) {
                throw new IOException("Server closed early, " + remaining + " bytes missing");
            }
            raf.write(buffer, 0, n);
            remaining = remaining - n;
        }
    }
}
```

- `in.read(buffer, 0, toRead)` อ่านได้**ไม่เกิน** toRead byte และคืน**จำนวนที่อ่านได้จริง** (อาจน้อยกว่า) หรือ -1 เมื่อจบ
- `raf.write(buffer, 0, n)` เขียนเฉพาะ n byte แรกของ buffer — ต้องใช้ `n` ไม่ใช่ `buffer.length` เพราะรอบสุดท้ายมักอ่านได้ไม่เต็ม buffer
- `RandomAccessFile` เปิดได้แบบ `"r"` (อ่าน) หรือ `"rw"` (อ่าน-เขียน) และ `seek(offset)` ข้ามไปยังตำแหน่งที่ต้องการ

3.11 Channel (NIO)

โค้ดจริงจากไฟล์ `FileSender.java` · บรรทัด 40–56

```
public static void sendWithNio(File file, long offset, long length, SocketChannel channel)
        throws IOException {
    try (FileChannel fileChannel = new RandomAccessFile(file, "r").getChannel()) {
        long position = offset;
        long remaining = length;

        while (remaining > 0) {
            // transferTo may send LESS than we asked, so we must repeat until all is sent
            long sent = fileChannel.transferTo(position, remaining, channel);
            if (sent <= 0) {
                throw new IOException("transferTo sent nothing");
            }
            position = position + sent;
            remaining = remaining - sent;
        }
    }
}
```

- `FileChannel` = ช่องทางสู่ไฟล์ · `SocketChannel` = ช่องทางสู่ TCP connection · `ServerSocketChannel` = ช่องทางรอรับ connection
- `transferTo(position, count, targetChannel)` ส่ง count byte จากไฟล์ไปยังช่องทางปลายทาง คืนจำนวนที่ส่งได้จริง ซึ่ง**อาจน้อยกว่า count** จึงต้องวนซ้ำ
- `if (sent <= 0) throw ...` กันการวนไม่รู้จบ ถ้า transferTo ไม่ขยับเลย
- ปิด `FileChannel` ก็ปิด `RandomAccessFile` ที่สร้างมันด้วย จึงใช้ try-with-resources กับ channel ได้

3.12 อื่น ๆ ที่พบในโค้ด

โค้ดจริงจากไฟล์ `ClientHandler.java` · บรรทัด 189–195

```
static boolean isAllowedName(String name) {
    if (name.contains("/") || name.contains("\\") || name.contains(":")
            || name.contains("..")) {
        return false;
    }
    return !Pattern.compile("\\s").matcher(name).find();
}
```

โค้ดจริงจากไฟล์ `Client.java` · บรรทัด 60–76

```
DownloadResult r = Downloader.download(host, port, filename, outFile,
        workers, useNio, true);
String check = "OK";
if (!r.verified) {
    check = "MISMATCH";
}
// Locale.ROOT: always a dot as the decimal mark, whatever the computer's language
System.out.printf(Locale.ROOT,
        "Done: %d bytes in %.3f s = %.1f MB/s, size+SHA-256 check: %s%n",
        r.sizeBytes, r.seconds, r.mbPerSec, check);
if (!r.verified) {
    // a damaged file must NOT look like a success to a script that checks the exit code
    System.err.println("Error: the downloaded file is DAMAGED (its size or SHA-256 "
            + "is different from the original). Do not use these numbers.");
    System.exit(2);
}
```

- **Regex:** `Pattern.compile("\\s").matcher(name).find()` = ในชื่อมีช่องว่าง/แท็บไหม (ใน Java ต้องเขียน backslash สองตัวใน string)
- **System.exit(n):** จบโปรแกรมพร้อมรหัส n (0 = ปกติ, ไม่ใช่ 0 = ผิดพลาด) เพื่อให้สคริปต์ตรวจได้ — Client ใช้ 0 / 1 (error) / 2 (ไฟล์เสียหาย)
- **ตัวแปร `null`:** ค่า "ยังไม่มี" เช่น `error = null` ใน RangeWorker แปลว่าไม่มีข้อผิดพลาด
- **Comment:** `// ...` บรรทัดเดียว · `/** ... */` ใช้อธิบาย class/เมธอด (คอมเมนต์ในโค้ดเขียนเป็นอังกฤษแบบง่าย)

ส่วนที่ 4 อธิบายทีละไฟล์

แต่ละไฟล์อธิบายตามลำดับเดียวกัน: **หน้าที่ → ใครเรียกใคร → สมาชิกสำคัญ → อ่านโค้ดทีละช่วง → จุดที่ควรรู้**
ลำดับไฟล์ไล่จากไฟล์พื้นฐานที่คนอื่นเรียกใช้ ไปหาไฟล์ที่เป็นจุดเริ่มต้นของโปรแกรม

Protocol.java

ใช้ร่วมกัน 59 บรรทัด · ค่าคงที่ของ protocol + อ่าน/ส่งข้อความ 1 บรรทัด

หน้าที่

เก็บ "กติกากลาง" ที่ทั้ง Server และ Client ใช้ร่วมกัน คือค่าคงที่ (พอร์ต, ขนาด buffer, ชื่อโหมด) และเมธอดอ่าน/ส่งข้อความ 1 บรรทัด จึงไม่มีโค้ดที่เขียนซ้ำกันสองฝั่ง

ใครเรียกใคร

ถูกเรียกจาก `Connection`, `ClientHandler`, `FileSender`, `RangeWorker`, `FileUtil` (ไม่เรียกไฟล์อื่นเลย)

สมาชิกสำคัญ

ชื่อ	ชนิด	ความหมาย
`DEFAULT_PORT`	int	พอร์ตเริ่มต้น 5000
`BUFFER_SIZE`	int	64 × 1024 = 65,536 byte ใช้เป็นขนาด byte[] ทั้งฝั่งอ่านไฟล์และอ่าน socket
`MODE_IO / MODE_NIO`	String	คำว่า `io` / `nio` ที่ส่งท้ายคำสั่ง GET
`readLine(in)`	String	อ่านทีละ byte จนเจอ `\n` คืนข้อความ หรือ `null` ถ้าอีกฝั่งปิด connection
`sendLine(out, text)`	void	ต่อ `\n` ท้ายข้อความแล้วส่ง พร้อม `flush()`

อ่านโค้ด

โค้ดจริงจากไฟล์ `Protocol.java` · บรรทัด 31–50

```java
public static String readLine(InputStream in) throws IOException {
    ByteArrayOutputStream bytes = new ByteArrayOutputStream();
    while (true) {
        int b = in.read();                  // read ONE byte (-1 means "connection closed")
        if (b == -1) {
            if (bytes.size() == 0) {
                return null;
            }
            break;
        }
        if (b == '\n') {
            break;
        }
        bytes.write(b);
        if (bytes.size() > 1024) {
            throw new IOException("Line is too long");
        }
    }
    // Turn the bytes into text only at the end (one character may use several bytes).
    return bytes.toString(StandardCharsets.UTF_8).trim();
```

- `in.read()` คืนค่า 0–255 ถ้ามี byte และ **-1** เมื่อ connection ปิด
- ถ้าปิดตอนที่ยังไม่ได้อ่านอะไรเลย คืน `null` (ฝั่ง server ใช้ตัดสินว่า client จากไป) แต่ถ้าปิดกลางบรรทัดก็ถือว่าบรรทัดจบ
- บรรทัดยาวเกิน 1024 byte โยน `IOException` กันคนส่งขยะมาจนหน่วยความจำเต็ม
- แปลง byte เป็นตัวอักษร (UTF-8) **ตอนท้ายสุดครั้งเดียว** เพราะ 1 ตัวอักษรไทยใช้ 3 byte ถ้าแปลงทีละ byte จะเพี้ยน

> **จุดที่ควรรู้: ทำไมไม่ใช้ BufferedReader**
> `BufferedReader` อ่านข้อมูลล่วงหน้าเป็นก้อนใหญ่ จึงอาจ "กลืน" byte ของไฟล์ที่ตามหลังบรรทัด `OK <length>` ไปด้วย ไฟล์ที่ได้จะเสียแบบหาสาเหตุยาก การอ่านทีละ byte ช้ากว่าเล็กน้อยแต่ใช้กับ header สั้น ๆ ไม่มีผลต่อเวลา (เวลาส่วนใหญ่อยู่ที่ตัวไฟล์) ส่วน `.trim()` ท้ายเมธอดตัด `\r` ที่อาจมาจาก client บน Windows

FileUtil.java

ใช้ร่วมกัน 38 บรรทัด · คำนวณ SHA-256 ของไฟล์

หน้าที่

คำนวณ **SHA-256** ของไฟล์ เป็น "ลายนิ้วมือ" ความยาว 64 ตัวอักษร ถ้าไฟล์สองไฟล์ลายนิ้วมือตรงกัน ถือว่าเหมือนกันทุก byte ใช้ตอนท้ายการดาวน์โหลดเพื่อยืนยันว่าไฟล์ไม่เสีย (เป็นข้อกำหนดของโจทย์: ตรวจ size/hash)

ใครเรียกใคร

ถูกเรียกจาก `ClientHandler.handleHash` (ฝั่ง server คำนวณของต้นฉบับ) และ `Downloader.download` (ฝั่ง client คำนวณของไฟล์ที่ได้) ใช้ `Protocol.BUFFER_SIZE`

อ่านโค้ด

โค้ดจริงจากไฟล์ `FileUtil.java` · บรรทัด 17–33

```
MessageDigest digest = MessageDigest.getInstance("SHA-256");
byte[] buffer = new byte[Protocol.BUFFER_SIZE];

// read the file piece by piece and feed every piece to the digest
int n = in.read(buffer);
while (n != -1) {
    digest.update(buffer, 0, n);
    n = in.read(buffer);
}

// turn the 32 result bytes into 64 hex characters, e.g. "3ccca89a..."
byte[] hash = digest.digest();
StringBuilder hex = new StringBuilder();
for (byte b : hash) {
    hex.append(String.format("%02x", b));
}
return hex.toString();
```

- อ่านไฟล์เป็นชิ้น ๆ ชิ้นละ 64 KB แล้วป้อนให้ `digest.update` จึงใช้ RAM น้อยแม้ไฟล์ใหญ่หลาย GB
- `String.format("%02x", b)` แปลง byte เป็นเลขฐาน 16 สองหลัก (`%02x` = เติม 0 ข้างหน้าให้ครบ 2 หลัก) 32 byte → 64 ตัวอักษร
- `try (FileInputStream ...)` ปิดไฟล์ให้เองแม้เกิดข้อผิดพลาด

> **จุดที่ควรรู้**
> SHA-256 ของไฟล์ใหญ่ใช้เวลาไม่น้อย (ไฟล์ 500 MB ราว 1–3 วินาที) จึง **ไม่นับรวมในเวลาที่วัด** (ดู `Downloader`) และ server คำนวณใหม่ทุกครั้งที่ถูกถาม HASH ไม่มี cache ซึ่งพอสำหรับงานนี้

Connection.java

Client 57 บรรทัด · TCP connection หนึ่งเส้น

หน้าที่

แทน TCP connection 1 เส้นฝั่ง client เก็บทั้ง `SocketChannel` (ที่ NIO ต้องใช้) และ `InputStream/OutputStream` (ที่ Traditional I/O ใช้) ไว้ที่เดียว จึงสลับ io/nio ได้โดยใช้ connection ชนิดเดียวกัน

ใครเรียกใคร

ถูกสร้างโดย `Downloader` (INFO/HASH), `RangeWorker` (GET), `Client` (LIST) และ `TestRunner`

สมาชิกสำคัญ

ชื่อ	หน้าที่
`Connection(host, port)`	ต่อไปยัง server ถ้าต่อไม่ได้โยน `IOException`
`sendLine / readLine`	ห่อ `Protocol.sendLine/readLine` ให้เรียกสั้นลง
`readReply()`	อ่าน 1 บรรทัดตอบกลับ ถ้า server ปิดหรือส่ง `ERROR ...` จะโยน exception ให้เลย (ผู้เรียกไม่ต้องเช็คเอง)

`close()`	ปิด channel (ทำให้ in/out ปิดตามไปด้วย)

อ่านโค้ด

โค้ดจริงจากไฟล์ `Connection.java` · บรรทัด 21–30

```
public Connection(String host, int port) throws IOException {
    try {
        channel = SocketChannel.open(new InetSocketAddress(host, port));
    } catch (UnresolvedAddressException e) {
        // Java gives an exception WITHOUT a message when the host name is wrong,
        // which would print "Error: null". We give a clear message instead.
        throw new IOException("Cannot find host \"" + host + "\" (check the name or IP)");
    }
    in = channel.socket().getInputStream();
    out = channel.socket().getOutputStream();
```

จุดที่ควรรู้
ชื่อ host ผิด (เช่น `localhots`) ทำให้ Java โยน `UnresolvedAddressException` ซึ่ง **ไม่มี message** ถ้าไม่จับ จอจะขึ้น `Error: null` โค้ดนี้จับแล้วแปลงเป็นข้อความอ่านรู้เรื่อง · `implements AutoCloseable` คือเหตุที่เขียน `try (Connection c = new Connection(...))` ได้ และ connection ถูกปิดเองทุกกรณี

Server.java

Server 79 บรรทัด · main ของ server: เปิดพอร์ต รับ connection

หน้าที่

จุดเริ่มของฝั่ง server ทำแค่ 3 อย่าง: (1) อ่านค่า port/โฟลเดอร์ (2) เปิดพอร์ตรอ (3) วนรับ client แล้วแจก **thread ใหม่ 1 ตัวต่อ 1 connection** เพื่อให้ 10 worker ของ client ถูกบริการพร้อมกัน

ใครเรียกใคร

เรียก `ClientHandler` · ถูกเรียกจาก command line (`java Server 5000 shared`) และ `TestRunner`

อ่านโค้ด: ลูปรับ client

โค้ดจริงจากไฟล์ `Server.java` · บรรทัด 55–69

```
while (true) {
    SocketChannel client = null;
    try {
        client = serverChannel.accept();            // waits here until a client connects
    } catch (IOException e) {
        // For example "Too many open files". One failed accept must not stop the
        // whole server: we write a message, wait a moment and try again.
        System.out.println("[server] accept failed: " + e.getMessage());
        pause(100);
        continue;
    }
    ClientHandler handler = new ClientHandler(client, folder);
    Thread thread = new Thread(handler);
    thread.start();                                 // handler.run() now runs in parallel
}
```

- `accept()` **หยุดรอ**จนกว่าจะมี client ต่อเข้ามา แล้วคืน `SocketChannel` ของ client คนนั้น

- `new Thread(handler).start()` ทำให้ `handler.run()` ทำงานคู่ขนาน ส่วน main thread วนกลับไป `accept()` รอคนต่อไปทันที
- ถ้า `accept()` ล้มเหลว (เช่น "Too many open files") จะพิมพ์ข้อความ รอ 0.1 วินาที แล้วรับต่อ **ไม่ปล่อยให้ server ตาย**

อ่านโค้ด: ตรวจค่าเริ่มต้น

โค้ดจริงจากไฟล์ `Server.java` · บรรทัด 29–32

```
if (port < 1 || port > 65535) {
    System.err.println("The port must be between 1 and 65535 (not " + port + ")");
    System.exit(1);
}
```

> **จุดที่ควรรู้**
> `getCanonicalFile()` แปลงชื่อโฟลเดอร์เป็นพาธเต็ม ใช้เป็นฐานตรวจชื่อไฟล์ · `System.exit(1)` = จบโปรแกรมด้วย exit code 1 (ผิดพลาด) เพื่อให้ script รู้ · พอร์ตถูกใช้อยู่แล้ว (เปิด server ซ้ำ) จะได้ข้อความ `Cannot open port ...` แทน stack trace · server **ไม่จำกัดจำนวน thread** (ดูข้อจำกัดในส่วนที่ 6)

ClientHandler.java

Server 209 บรรทัด · คุยกับ client หนึ่งราย (1 thread ต่อ 1 connection)

หน้าที่

คุยกับ client 1 ราย: อ่านคำสั่งทีละบรรทัด ตรวจความถูกต้อง แล้วตอบ `LIST / INFO / HASH / GET` หรือ `ERROR` จนกว่า client จะปิด connection

ใครเรียกใคร

ถูกสร้างโดย `Server` · เรียก `Protocol`, `FileUtil.sha256`, `FileSender`

สมาชิกสำคัญ

ชื่อ	หน้าที่
`idleTimeoutMs`	60,000 ms: ถ้า client เงียบเกินนี้ server ตัด connection (ไม่ใช่ final เพื่อให้ TestRunner ตั้งให้สั้นตอนทดสอบ)
`run()`	ลูปหลักของ thread: อ่านบรรทัด → `handleCommand` ซ้ำไปจนอ่านได้ `null`
`handleCommand`	แยกคำแรก แล้วเรียกเมธอดที่ตรงกัน ไม่รู้จัก → `ERROR 400`
`handleList / Info / Hash / Get`	เมธอดละ 1 คำสั่ง
`isAllowedName`	กฎเดียวกันที่ใช้ทั้งใน LIST และตอนหาไฟล์ (กัน path traversal)
`findFile`	ชื่อผ่านกฎ + เป็นไฟล์จริงในโฟลเดอร์ที่แชร์ → คืน `File` มิฉะนั้น `null`

อ่านโค้ด: ลูปของ thread

โค้ดจริงจากไฟล์ `ClientHandler.java` · บรรทัด 34–59

```
public void run() {
    try {
        in = channel.socket().getInputStream();
        out = channel.socket().getOutputStream();
        channel.socket().setSoTimeout(idleTimeoutMs);    // only affects READING commands

        while (true) {
            String line = Protocol.readLine(in);
            if (line == null) {
                break;                      // the client closed the connection
            }
            handleCommand(line);
        }
    } catch (SocketTimeoutException e) {
        System.out.println("[server] closing a connection that was idle for too long");
    } catch (IOException e) {
        // normal when a client disconnects in the middle of a transfer
        System.out.println("[server] connection ended: " + e.getMessage());
    } finally {
        try {
            channel.close();                // always close the connection at the end
        } catch (IOException e) {
            // nothing more we can do
        }
    }
}
```

- `catch` ของ `SocketTimeoutException` ต้องมาก่อน `IOException` เพราะมันเป็นลูกของ IOException (ถ้าสลับ จะไม่มีวันเข้า catch แรก)
- `finally` ปิด channel เสมอ ไม่ว่าจะจบแบบปกติหรือมี error
- `setSoTimeout` มีผลกับ **การอ่าน**เท่านั้น ไม่กระทบการส่งไฟล์ขนาดใหญ่

อ่านโค้ด: ตรวจคำสั่ง GET (หัวใจของความปลอดภัย)

โค้ดจริงจากไฟล์ `ClientHandler.java` · บรรทัด 166–170

```
long size = file.length();
if (offset < 0 || length < 0 || offset > size || length > size - offset) {
    Protocol.sendLine(out, "ERROR 416 Range out of bounds (file size is " + size + ")");
    return;
}
```

ลำดับการตรวจใน `handleGet` คือ จำนวนคำ → มีไฟล์ไหม → offset/length เป็นตัวเลขไหม → โหมดถูกไหม → ช่วงอยู่ในไฟล์ไหม ผิดข้อไหนตอบ error ข้อนั้นและ **ไม่ปิด connection** client ส่งคำสั่งต่อได้

รหัส error

รหัส	เมื่อไหร่	ตัวอย่างที่ server ตอบ
400	คำสั่งผิดรูป / ไม่รู้จัก / ตัวเลขไม่ใช่ตัวเลข / โหมดไม่ใช่ io|nio	`ERROR 400 Offset and length must be numbers`
404	ไม่พบไฟล์ หรือชื่อไม่ผ่าน `isAllowedName`	`ERROR 404 File not found`
416	ช่วงที่ขอเกินขนาดไฟล์ หรือเป็นค่าลบ	`ERROR 416 Range out of bounds (file size is 44)`

ทำไมเขียน `length > size - offset` ไม่ใช่ `offset + length > size`
ถ้า client ส่ง `offset=9223372036854775807` (ค่ามากสุดของ `long`) แล้วบวก length เข้าไป ผลจะ **ล้น (overflow)** กลายเป็นเลขติดลบ การตรวจแบบบวกจึงผ่านโดยผิดพลาด ส่วนการลบ `size - offset` ปลอดภัยเพราะตรวจ `offset > size` ไปก่อนแล้ว

จุดที่ควรรู้
`toUpperCase(Locale.ROOT)` : ถ้าเครื่องตั้งภาษาตุรกี `"info".toUpperCase()` จะได้ `İNFO` (ตัว i มีจุด) ทำให้ไม่รู้จักคำสั่ง จึงบังคับกฎภาษาอังกฤษ · ชื่อไฟล์ที่มี `/ \ : ..` หรือช่องว่างถูกปฏิเสธ เพราะคำสั่งถูกแยกคำด้วยช่องว่าง ชื่อ `my file.txt` จะกลายเป็น 2 คำและหาไม่เจอ จึงไม่แสดงใน LIST ด้วย (LIST กับ GET ใช้กฎเดียวกัน)

FileSender.java

Server 57 บรรทัด · ส่งข้อมูลช่วงที่ขอ: แบบ io และแบบ nio

หน้าที่

ไฟล์ที่ทำให้เกิดการทดลอง มี 2 เมธอดที่ส่งข้อมูลช่วง [offset, offset+length) ออกทาง socket ด้วยวิธีต่างกัน

ใครเรียกใคร

ถูกเรียกจาก `ClientHandler.handleGet` หลังส่งบรรทัด `OK <length>` ไปแล้ว

วิธีที่ 1: Traditional I/O (io)

โค้ดจริงจากไฟล์ `FileSender.java` · บรรทัด 16–34

```
public static void sendWithStreams(File file, long offset, long length, OutputStream out)
        throws IOException {
    try (RandomAccessFile raf = new RandomAccessFile(file, "r")) {
        raf.seek(offset);                                   // jump to the start of our part
        byte[] buffer = new byte[Protocol.BUFFER_SIZE];
        long remaining = length;                            // bytes we still have to send

        while (remaining > 0) {
            int toRead = (int) Math.min(buffer.length, remaining);
            int n = raf.read(buffer, 0, toRead);            // n = how many bytes we really got
            if (n == -1) {
                throw new IOException("File ended too early");
            }
            out.write(buffer, 0, n);
            remaining = remaining - n;
        }
        out.flush();
    }
}
```

ข้อมูลเดินทาง: **ไฟล์ → (kernel) → byte[] ในโปรแกรม → (kernel) → socket** คือถูกคัดลอกเข้า-ออกพื้นที่ของโปรแกรมหนึ่งรอบ `raf.read` อาจได้น้อยกว่าที่ขอ จึงใช้ตัวแปร `n` และลูป `while` นับ `remaining`

วิธีที่ 2: NIO transferTo (nio)

โค้ดจริงจากไฟล์ `FileSender.java` · บรรทัด 40–56

```
public static void sendWithNio(File file, long offset, long length, SocketChannel channel)
        throws IOException {
    try (FileChannel fileChannel = new RandomAccessFile(file, "r").getChannel()) {
        long position = offset;
        long remaining = length;

        while (remaining > 0) {
            // transferTo may send LESS than we asked, so we must repeat until all is sent
            long sent = fileChannel.transferTo(position, remaining, channel);
            if (sent <= 0) {
                throw new IOException("transferTo sent nothing");
            }
            position = position + sent;
            remaining = remaining - sent;
        }
    }
}
```

ข้อมูลเดินทาง: **ไฟล์ → socket** โดยระบบปฏิบัติการทำเอง (บน Linux คือ system call `sendfile`) ไม่ผ่าน `byte[]` ของโปรแกรม นี่คือ zero-copy ฝั่งส่งของจริง `transferTo` อาจส่งได้น้อยกว่าที่ขอ จึงต้องวนซ้ำจนครบ และถ้าได้ 0 ต้องโยน error ไม่งั้นลูปไม่จบ

> **จุดที่ควรรู้**
> ทั้งสองวิธีส่ง **จำนวน byte เท่ากันเป๊ะ**ตามที่ประกาศใน `OK <length>` client จึงรู้ว่าต้องอ่านกี่ byte (ไม่มีตัวปิดท้าย) · `try (RandomAccessFile ...)` ปิดไฟล์ให้ทุกกรณี · ไฟล์ถูกเปิดด้วยโหมด `"r"` (อ่านอย่างเดียว) จึงไม่มีทางแก้ไฟล์ต้นฉบับ

`Client.java`

`Client` 153 บรรทัด · main ของ client: อ่านคำสั่งจาก command line

หน้าที่

จุดเริ่มของฝั่ง client อ่านคำสั่งจาก command line แล้วส่งต่อให้ไฟล์ที่ทำงานจริง (`Downloader`, `Benchmark`) ไฟล์นี้ **ไม่มีตรรกะดาวน์โหลดเอง**

คำสั่งที่รองรับ

คำสั่ง	ทำอะไร	เรียกต่อไปที่
`list`	แสดงไฟล์ทั้งหมดบน server	`Client.list` (ส่ง LIST)
`info <file>`	แสดงขนาดไฟล์	`Downloader.getFileSize`
`download <file> <out> [workers] [io|nio]`	ดาวน์โหลดด้วย worker (ค่าเริ่มต้น 10, ช่วง 1–100) แล้วตรวจ	`Downloader.download`
`benchmark <file> [runs]`	ทดลอง io/nio × 1/10 worker	`Benchmark.run`

อ่านโค้ด: คำสั่ง download และ exit code

โค้ดจริงจากไฟล์ `Client.java` · บรรทัด 60–76

```
DownloadResult r = Downloader.download(host, port, filename, outFile,
        workers, useNio, true);
String check = "OK";
if (!r.verified) {
    check = "MISMATCH";
}
// Locale.ROOT: always a dot as the decimal mark, whatever the computer's language
System.out.printf(Locale.ROOT,
        "Done: %d bytes in %.3f s = %.1f MB/s, size+SHA-256 check: %s%n",
        r.sizeBytes, r.seconds, r.mbPerSec, check);
if (!r.verified) {
    // a damaged file must NOT look like a success to a script that checks the exit code
    System.err.println("Error: the downloaded file is DAMAGED (its size or SHA-256 "
            + "is different from the original). Do not use these numbers.");
    System.exit(2);
}
```

exit code	ความหมาย
0	สำเร็จ และไฟล์ตรวจผ่าน
1	ผิดพลาด (ต่อ server ไม่ได้, ไม่พบไฟล์, worker ล้ม, อาร์กิวเมนต์ผิด)
2	ดาวน์โหลดจบแต่ไฟล์ **เสีย** (ขนาดหรือ SHA-256 ไม่ตรง)

อ่านโค้ด: แปลงตัวเลขอย่างปลอดภัย

โค้ดจริงจากไฟล์ `Client.java` · บรรทัด 103–109

```
static int parseNumber(String text, String name) {
    try {
        return Integer.parseInt(text);
    } catch (NumberFormatException e) {
        throw new IllegalArgumentException(name + " must be a number (not \"" + text + "\")");
    }
}
```

จุดที่ควรรู้
ผู้ใช้ลืมใส่จำนวน worker แล้วพิมพ์ `nio` ในตำแหน่งนั้น จะได้ข้อความ `workers must be a number (not "nio")` แทนที่จะเป็น stack trace · คำสั่ง `io|nio` พิมพ์ผิด (เช่น `nioo`) จะ **ไม่ถูกเงียบ ๆ ตีความเป็น io** เพราะจะทำให้ผลทดลองผิดวิธี · `catch (Exception e)` ตัวเดียวท้าย main รวมทุก error เป็นรูปแบบ `Error: ...` + exit 1 · คำสั่ง `download` ถาม INFO ไม่ใช่ LIST (LIST ใช้แสดงรายชื่อไฟล์เท่านั้น)

Downloader.java

Client 93 บรรทัด · งานดาวน์โหลดทั้งหมด: แบ่งช่วง สั่ง worker ตรวจไฟล์

หน้าที่

ผู้จัดการงานดาวน์โหลด ถามขนาดไฟล์ → สร้างไฟล์ปลายทางขนาดเต็ม → แบ่งช่วงให้ worker → สั่งเริ่ม → รอจบ → ตรวจไฟล์ → คืนผล ตรงกับข้อกำหนดโจทย์ "แบ่งไฟล์เป็น 10 ช่วง ใช้ 10 worker เขียนลงไฟล์เดียวตาม offset"

ใครเรียกใคร

ถูกเรียกจาก `Client`, `Benchmark`, `TestRunner` · เรียก `Connection`, `RangeWorker`, `FileUtil`, `DownloadResult`

ขั้นตอนใน `download()`

1. `getFileSize` ส่ง `INFO` ได้ขนาดไฟล์ `size`
2. `setLength(size)` สร้างไฟล์ปลายทางให้ใหญ่เท่าต้นฉบับก่อน
3. คำนวณช่วงของแต่ละ worker แล้ว `start()`
4. `join()` รอทุก worker จบ แล้วหยุดจับเวลา
5. ถ้ามี worker ใด error → ลบไฟล์ปลายทางทิ้ง แล้วโยน `IOException`
6. ตรวจขนาด + SHA-256 เทียบกับที่ server ตอบ (**ไม่นับในเวลา**)

อ่านโค้ด: แบ่งช่วงและสั่ง worker

โค้ดจริงจากไฟล์ `Downloader.java` · บรรทัด 49–68

```
long chunk = size / workers;
RangeWorker[] list = new RangeWorker[workers];

long startTime = System.nanoTime();
for (int i = 0; i < workers; i++) {
    long offset = i * chunk;
    long length = chunk;
    if (i == workers - 1) {
        length = size - offset;
    }
    list[i] = new RangeWorker(i, host, port, filename, outFile,
            offset, length, useNio, verbose);
    list[i].start();                              // the worker starts running in parallel
}

// 4) wait until every worker has finished
for (int i = 0; i < workers; i++) {
    list[i].join();
}
double seconds = (System.nanoTime() - startTime) / 1e9;
```

- `chunk = size / workers` เป็นการหารจำนวนเต็ม เศษที่เหลือ (`size % workers`) ถูกโยนให้ worker **ตัวสุดท้าย**: `length = size - offset`
- `start()` ในลูปแรกทำให้ทุก worker เริ่มพร้อมกัน แล้วลูปที่สองค่อย `join()` ทีละตัว (ถ้า join ในลูปเดียวกับ start จะกลายเป็นทำทีละตัว!)
- `i * chunk`: `i` เป็น `int` แต่ `chunk` เป็น `long` ผลจึงคำนวณเป็น `long` ไม่ล้นแม้ไฟล์เกิน 2 GB

ตัวอย่าง size = 1,000,003 / 10 workers	ค่า
chunk	100,000
worker 0 … 8	offset = i × 100,000 · length = 100,000
worker 9 (ตัวสุดท้าย)	offset = 900,000 · length = 100,003 (ได้เศษ 3 byte)

size = 5 (น้อยกว่า worker)	chunk = 0 · worker 0–8 ได้ length 0 · worker 9 ได้ทั้ง 5 byte

อ่านโค้ด: ตรวจผล

โค้ดจริงจากไฟล์ `Downloader.java` · บรรทัด 87–91

```
boolean sameSize = (outFile.length() == size);
String myHash = FileUtil.sha256(outFile);
String serverHash = getServerHash(host, port, filename);
result.verified = sameSize && myHash.equalsIgnoreCase(serverHash);
return result;
```

> **จุดที่ควรรู้**
> **ทำไมต้อง `setLength(size)` ก่อน:** worker เขียนตามตำแหน่ง (`seek`) ถ้าไฟล์ยังเล็กอยู่ NIO `transferFrom` จะ **ไม่ถ่ายอะไรเลย** เมื่อตำแหน่งเกินขนาดไฟล์ปัจจุบัน · **ทำไมไม่ต้องใช้ lock:** แต่ละ worker เขียนคนละช่วงไม่ทับกัน และ `join()` รับประกันว่าค่า `error` ของ worker ที่ main thread อ่านเป็นค่าล่าสุด · ถ้า worker ล้ม ไฟล์ปลายทางที่ใหญ่เต็มขนาดแต่มีช่องโหว่ จะถูกลบ เพื่อไม่ให้ใครเข้าใจผิดว่าเป็นไฟล์สมบูรณ์

DownloadResult.java

Client 10 บรรทัด · กล่องเก็บผลของการดาวน์โหลดหนึ่งครั้ง

หน้าที่

"กล่อง" เก็บผลของการดาวน์โหลด 1 ครั้ง มีแต่ฟิลด์ ไม่มีเมธอด เพื่อให้ `download()` คืนหลายค่าพร้อมกันได้

โค้ดจริงจากไฟล์ `DownloadResult.java` · บรรทัด 6–9

```
public long sizeBytes;       // size of the file
public double seconds;       // time used by the workers
public double mbPerSec;      // throughput in MB/s
public boolean verified;     // true if size and SHA-256 are the same as the original
```

ฟิลด์	ความหมาย
`sizeBytes`	ขนาดไฟล์ (byte)
`seconds`	เวลาที่ worker ใช้ (วินาที) นับตั้งแต่ก่อน start จนหลัง join ครบ **ไม่รวมการตรวจ hash**
`mbPerSec`	throughput = (size / 1024 / 1024) ÷ seconds หน่วย MB/s
`verified`	true เมื่อขนาดและ SHA-256 ตรงกับต้นฉบับ

> **จุดที่ควรรู้**
> ไฟล์ขนาด 0 byte จะได้ `seconds` เล็กมาก และ `mbPerSec` เป็น 0 หรือ Infinity ได้ ไม่ถือว่าผิด (ไม่มีข้อมูลให้วัด) ใน TestRunner ไฟล์ว่างถูกทดสอบเรื่อง ความถูกต้อง ไม่ใช่ความเร็ว

RangeWorker.java

Client 122 บรรทัด · worker 1 ตัว: connection ของตัวเอง ช่วงของตัวเอง

หน้าที่

worker 1 ตัว = **1 thread + 1 connection + 1 ช่วงของไฟล์** ส่ง `GET file offset length mode` รับข้อมูล แล้วเขียนลงไฟล์ปลายทางที่ตำแหน่ง `offset` ตรงตามโจทย์ "แต่ละ worker มี connection, offset และ length ของตัวเอง"

ใครเรียกใคร

ถูกสร้างและสั่ง `start()` โดย `Downloader` · เรียก `Connection`, `Protocol`

อ่านโค้ด: `run()`

โค้ดจริงจากไฟล์ `RangeWorker.java` · บรรทัด 44–71

```
public void run() {
    long startTime = System.nanoTime();

    try (Connection c = new Connection(host, port)) {
        // 1) ask the server for our part of the file
        String mode = Protocol.MODE_IO;
        if (useNio) {
            mode = Protocol.MODE_NIO;
        }
        c.sendLine("GET " + filename + " " + offset + " " + length + " " + mode);

        // 2) the server answers "OK <length>" (or "ERROR ...")
        String reply = c.readReply();
        String[] words = reply.split(" ");
        long announced = Long.parseLong(words[1]);
        if (announced != length) {
            throw new IOException("Server announced " + announced
                    + " bytes, expected " + length);
        }

        // 3) receive the bytes and write them into the file
        if (useNio) {
            receiveWithNio(c);
        } else {
            receiveWithStreams(c);
        }
    } catch (Exception e) {
        error = e.toString();
```

- `extends Thread` + เมธอด `run()` คือโค้ดที่ทำงานใน thread ใหม่หลังถูกเรียก `start()`
- ตรวจว่า server ประกาศความยาวตรงกับที่ขอ (`announced != length`) ก่อนรับข้อมูล
- `catch (Exception e)`: error ใน thread ลูกโยนกลับไปให้ main ไม่ได้ จึง **เก็บข้อความไว้ในฟิลด์ `error`** แล้ว main อ่านหลัง `join()`

รับข้อมูล: io

โค้ดจริงจากไฟล์ `RangeWorker.java` · บรรทัด 83–98

```
private void receiveWithStreams(Connection c) throws IOException {
    try (RandomAccessFile raf = new RandomAccessFile(outFile, "rw")) {
        raf.seek(offset);                                   // jump to our part of the file
        byte[] buffer = new byte[Protocol.BUFFER_SIZE];
        long remaining = length;                            // bytes still missing

        while (remaining > 0) {
            int toRead = (int) Math.min(buffer.length, remaining);
            int n = c.in.read(buffer, 0, toRead);
            if (n == -1) {
                throw new IOException("Server closed early, " + remaining + " bytes missing");
            }
            raf.write(buffer, 0, n);
            remaining = remaining - n;
        }
    }
```

รับข้อมูล: nio

โค้ดจริงจากไฟล์ `RangeWorker.java` · บรรทัด 106–120

```
private void receiveWithNio(Connection c) throws IOException {
    try (FileChannel fileChannel = new RandomAccessFile(outFile, "rw").getChannel()) {
        long position = offset;
        long remaining = length;

        while (remaining > 0) {
            // transferFrom may receive LESS than we asked, so we repeat until all arrived
            long received = fileChannel.transferFrom(c.channel, position, remaining);
            if (received <= 0) {
                throw new IOException("Server closed early, " + remaining + " bytes missing");
            }
            position = position + received;
            remaining = remaining - received;
        }
    }
```

จุดที่ควรรู้: `transferFrom` จาก socket ไม่ใช่ zero-copy แท้
เมื่อต้นทางเป็น socket JDK ยังต้องอ่านทีละก้อนเล็ก (ราว 8 KB) เข้า buffer ชั่วคราวแล้วเขียนลงไฟล์ จึงใช้ system call มากกว่า buffer 64 KB ของวิธี io ราว 8 เท่า นี่คือเหตุผลหนึ่งที่ **nio ไม่ได้ชนะเสมอไป** (ฝั่งส่ง `transferTo` เป็น zero-copy จริง แต่ฝั่งรับไม่ใช่) ควรเขียนข้อสังเกตนี้ในรายงานผลการทดลอง

แต่ละ worker เปิดไฟล์ปลายทางเอง
ทุก worker เปิด `RandomAccessFile(outFile, "rw")` ของตัวเอง (ไม่ใช้ตัวร่วม) แล้ว `seek` ไปที่ offset ของตัวเอง จึงไม่มีตำแหน่งอ่าน/เขียนร่วมกัน ไม่ต้อง synchronized

Benchmark.java

Client 115 บรรทัด · รันการทดลองและเก็บผลเป็นตาราง/CSV

หน้าที่

ทำการทดลองตามโจทย์: **io / nio × 1 / 10 worker × อย่างน้อย 3 รอบ** พิมพ์ตารางเวลา, MB/s, ผลตรวจ แล้วบันทึกเป็น `benchmark_results.csv` เพื่อเปิดใน Excel ทำกราฟ

ลำดับที่รัน

ลำดับ	mode	workers	รอบ
1	io	1	อุ่นเครื่อง 1 ครั้ง (ไม่นับ) + นับ runs ครั้ง
2	io	10	เหมือนกัน
3	nio	1	เหมือนกัน
4	nio	10	เหมือนกัน

อ่านโค้ด: ลูปทดลอง

โค้ดจริงจากไฟล์ `Benchmark.java` · บรรทัด 64–79

```
for (int run = 1; run <= runs; run++) {
    DownloadResult r = downloadOnce(host, port, filename, tempFile,
            workers, useNio);
    totalTime = totalTime + r.seconds;
    totalSpeed = totalSpeed + r.mbPerSec;

    String check = "OK";
    if (!r.verified) {
        check = "MISMATCH";
    }
    System.out.printf(Locale.ROOT, "%-6s %-8d %-4d %-10.3f %-10.1f %-9s%n",
            mode, workers, run, r.seconds, r.mbPerSec, check);
    csv.println(mode + "," + workers + "," + run + ","
            + String.format(Locale.ROOT, "%.3f", r.seconds) + ","
            + String.format(Locale.ROOT, "%.1f", r.mbPerSec) + ","
            + r.verified);
```

- **Warm-up:** ดาวน์โหลดรอบแรกของแต่ละกรณีช้ากว่าปกติ (JVM ต้องโหลด/คอมไพล์โค้ด และไฟล์ยังไม่อยู่ใน cache ของ OS) จึงทำ 1 ครั้งก่อนโดยไม่นับ แต่ต้องถูกต้อง
- **ไฟล์ใหม่ทุกครั้ง:** `tempFile.delete()` ก่อนดาวน์โหลด ทุกรอบเริ่มจากสภาพเดียวกัน
- ถ้าผลตรวจเป็น MISMATCH **หยุดทันที** เพราะตัวเลขความเร็วของไฟล์เสียไม่มีความหมาย (แถวที่เสียยังถูกเขียนลง CSV)
- `finally` ปิด CSV และลบไฟล์ชั่วคราวเสมอ

บั๊กที่พบและแก้แล้ว: เครื่องที่ตั้งภาษาเยอรมัน/ฝรั่งเศส
เดิมใช้ `%.3f` ธรรมดา ซึ่งใช้ **เครื่องหมายจุลภาค** เป็นทศนิยมตาม locale ของเครื่อง (`0,006`) ทำให้คอลัมน์ใน CSV เลื่อน แก้โดยใส่ `Locale.ROOT` ทุกจุดที่จัดรูปแบบตัวเลข และเพิ่ม test `germanCsvTest` ตั้ง locale เป็น GERMANY แล้วตรวจว่าทุกแถวมี 6 ช่องพอดี

จุดที่ควรรู้
ผลต่อรอบจะแกว่ง เพราะมี cache ของ OS, โหลดเครื่อง และ localhost ไม่ใช่เครือข่ายจริง ควรใช้ไฟล์ใหญ่ (200–500 MB) และดูค่าเฉลี่ย · ถ้าไฟล์เล็กมาก ค่าใช้จ่ายในการสร้าง thread และ connection 10 เส้นอาจมากกว่าเวลาโอนข้อมูลจริง ทำให้ 10 worker ช้ากว่า 1 worker

MakeTestFile.java

เครื่องมือ 44 บรรทัด · สร้างไฟล์สุ่มขนาดใหญ่สำหรับทดลอง

หน้าที่

เครื่องมือเล็ก ๆ สร้างไฟล์สุ่มขนาดที่ต้องการ (MB) ใช้ได้ทั้ง Windows / Linux / macOS โดยไม่ต้องใช้สิทธิ์ผู้ดูแล เช่น `java MakeTestFile shared/test.bin 500`

โค้ดจริงจากไฟล์ `MakeTestFile.java` · บรรทัด 30–40

```
Random random = new Random(12345);           // a fixed number: same content every time
byte[] block = new byte[1024 * 1024];        // 1 MB
FileOutputStream out = new FileOutputStream(file);
for (int i = 1; i <= sizeMb; i++) {
    random.nextBytes(block);                 // fill the block with random bytes
    out.write(block);
    if (i % 100 == 0) {
        System.out.println("  written " + i + " MB ...");
    }
}
out.close();
```

- `new Random(12345)` ใช้ seed คงที่ จึงได้เนื้อไฟล์เหมือนเดิมทุกครั้ง (ทำซ้ำการทดลองได้)
- เขียนทีละ 1 MB ไม่สร้าง byte[] ขนาดทั้งไฟล์ จึงสร้างไฟล์ GB ได้โดยไม่ใช้ RAM มาก
- `mkdirs()` สร้างโฟลเดอร์ปลายทางให้ถ้ายังไม่มี

จุดที่ควรรู้
ถ้าพิมพ์ขนาดไม่ใช่ตัวเลข จะเกิด `NumberFormatException` แบบ stack trace (ยังไม่ได้ทำข้อความสวย เพราะเป็นแค่เครื่องมือช่วย ไม่ใช่ส่วนของระบบ) ไฟล์สุ่มบีบอัดไม่ได้ จึงเหมาะกับการวัดความเร็วจริง

TestRunner.java

เครื่องมือ 740 บรรทัด · test case อัตโนมัติ 84 ข้อ

หน้าที่

ชุดทดสอบอัตโนมัติ 84 ข้อ รันด้วย `java TestRunner` ทำเองทั้งหมด: สร้างไฟล์ทดสอบในโฟลเดอร์ชั่วคราว, เปิด server ในโปรแกรมเดียวกัน (พอร์ต 5055), รันการตรวจแล้วพิมพ์ PASS/FAIL ลบไฟล์ทิ้ง และคืน exit code 0 (ผ่านหมด) หรือ 1 (มีข้อไม่ผ่าน)

กลุ่มของ test

กลุ่ม	ตรวจอะไร
1 Protocol	LIST / INFO / HASH / GET และทุก error (400/404/416, path traversal, offset ใหญ่สุดของ long, ฯลฯ)
2 Download	ไฟล์ 7 ขนาด × io/nio × 1/3/10 worker ตรวจด้วย SHA-256 เทียบกับต้นฉบับ
3 Concurrency	4 client (40 connection) โหลดพร้อมกัน
4 Robustness	ไม่มีไฟล์, client ตัดกลางทาง, ไฟล์ปลายทางเก่า, connection ที่เงียบ, กฎภาษาตุรกี
5 Client	คำว่า io/nio บน command line, ตัวเลขผิด, benchmark และ CSV, locale เยอรมัน
6 Failure	server ปลอมที่ทำตัวผิดตั้งใจ (ส่ง byte เสีย, ปิดก่อนครบ) และ exit code ของ `Client` จริง

ไฟล์ทดสอบ 7 ขนาด และเหตุผล

โค้ดจริงจากไฟล์ `TestRunner.java` · บรรทัด 45–47

```
static final String[] NAMES = {"empty.bin", "one.bin", "tiny.bin", "ten.bin",
                               "odd.bin", "mid.bin", "big.bin"};
static final long[] SIZES = {0, 1, 9, 10, 1000003, 5 * 1024 * 1024 + 7, 8 * 1024 * 1024 + 7};
```

ขนาด	ทดสอบกรณีอะไร
0 byte	ไฟล์ว่าง: ทุก worker ได้ length 0
1, 9 byte	เล็กกว่าจำนวน worker: บาง worker ได้ 0 byte
10 byte	worker ละ 1 byte พอดี
1,000,003	หาร 10 ไม่ลงตัว: worker สุดท้ายต้องรับเศษ
5 MB + 7 / 8 MB + 7	ใหญ่กว่า buffer หลายเท่า และไม่ลงตัว

ผลที่รันจริง
ผ่าน **84/84** ข้อ (รันหลายรอบ) และคอมไพล์ผ่านด้วย `javac --release 11` · รันซ้ำได้บนเครื่องของคุณด้วยคำสั่งเดียว

ส่วนที่ 5 ตรวจโค้ดเทียบกับโจทย์

ตรวจทีละข้อของใบงาน (ไฟล์ Assignment_Multithread_File_Download.pdf) กับโค้ดที่ส่งมอบจริง ผลคือ **ข้อกำหนดด้านโปรแกรมครบทุกข้อ** ส่วนที่เหลือเป็นงานที่ต้องทำเองบนเครื่องของคุณ คือรันการทดลองเก็บตัวเลขจริงและเขียนคำอธิบายผลในรายงาน

ข้อกำหนดของโจทย์	สถานะ	อยู่ตรงไหน / หลักฐาน
Client/Server ผ่าน TCP, command-line	ครบ	`Server`, `Client` ใช้ `SocketChannel`
Server รองรับหลาย connection พร้อมกัน	ครบ	1 thread ต่อ 1 connection (`Server` → `ClientHandler`) · test กลุ่ม 3: 40 connection พร้อมกัน
Client แบ่งไฟล์เป็น 10 ช่วง ใช้ 10 workers	ครบ	ค่าเริ่มต้น workers = 10 · `Downloader.download` สร้าง `RangeWorker[10]`
แต่ละ worker มี connection / offset / length ของตัวเอง	ครบ	`RangeWorker` เปิด `Connection` เอง และรับ offset/length ใน constructor
ช่วงข้อมูลไม่ซ้อนกัน	ครบ	`offset = i × chunk`, ตัวสุดท้ายรับเศษ · test ไฟล์ 7 ขนาด ตรวจด้วย SHA-256
เขียนลง final file ตาม offset (แยก file handle ต่อ worker)	ครบ	ทุก worker เปิด `RandomAccessFile` ของตัวเอง แล้ว `seek(offset)`
Protocol: LIST	ครบ	`FILE <name> <size>` ต่อบรรทัด จบด้วย `END`
Protocol: INFO	ครบ	`SIZE <bytes>` หรือ `ERROR 404`
Protocol: GET <file> <offset> <length>	ครบ	`OK <length>` แล้วตามด้วย byte ดิบ · มีโหมด `io|nio` เป็นตัวเลือกท้ายคำสั่ง
Protocol: ERROR <code> <message>	ครบ	400 / 404 / 416
Client ขอรายชื่อ/ขนาดไฟล์ก่อนเริ่ม	ครบ	`download` ถาม INFO ก่อน · `list` แสดงรายชื่อ (LIST เป็นคำสั่งแยก)
Traditional I/O (InputStream/OutputStream, RandomAccessFile)	ครบ	`FileSender.sendWithStreams`, `RangeWorker.receiveWithStreams`
NIO transferTo / transferFrom	ครบ	`FileSender.sendWithNio`, `RangeWorker.receiveWithNio`
ตรวจ size/hash หลังดาวน์โหลด	ครบ	ตรวจขนาด + SHA-256 เทียบกับ `HASH` จาก server · ไม่ผ่าน = exit code 2
ทดลอง 1 และ 10 workers ทั้งสองวิธี อย่างน้อย 3 รอบ	ครบ	`java Client localhost 5000 benchmark test.bin 3` · เตือนถ้า runs < 3
บันทึกเวลารวมและ throughput (MB/s)	ครบ	ตารางบนหน้าจอ + `benchmark_results.csv`
ใช้ไฟล์เดียวกัน สภาพแวดล้อมเดียวกัน	ครบ	benchmark ใช้ไฟล์เดียว รันทุกกรณีในโปรแกรมเดียว

อธิบายว่าทำไม 10 workers ไม่เร็วขึ้น 10 เท่า / NIO ไม่ชนะเสมอไป	**ต้องทำเอง**	ต้องเขียนในรายงานด้วยตัวเลขของคุณ · มีเนื้อหาให้ใน `README_THAI.txt` หัวข้อ 7 และสรุปในส่วนที่ 7 ของ PDF นี้
ระบุข้อจำกัดของการทดสอบบน localhost	**ต้องทำเอง**	ต้องเขียนในรายงาน · มีรายการให้ใน README หัวข้อ 7 (page cache, loopback, storage cache)
ส่ง source code + คำสั่ง compile/run	**ครบ**	`HOW_TO_RUN.txt`, README ทั้งสองภาษา

> **ข้อสังเกตเรื่อง LIST กับ INFO**
> แนวทางข้อ 1 ของโจทย์พูดว่า "ขอรายชื่อ/ข้อมูลไฟล์และขนาด" โปรแกรมนี้ให้ `download` ใช้ `INFO` (เพราะรู้ชื่อไฟล์อยู่แล้ว) ส่วน `LIST` ทำงานครบถ้วนแต่เรียกผ่านคำสั่ง `list` ถ้าอาจารย์ต้องการให้ดาวน์โหลดเรียก LIST ก่อนทุกครั้ง เพิ่มได้ง่าย แต่ไม่จำเป็นตามข้อความของโจทย์ (ที่เขียนว่า "รายชื่อ/ข้อมูลไฟล์")

5.1 สิ่งที่ต้องทำเองก่อนส่งงาน

1. คอมไพล์และรัน `java TestRunner` บนเครื่องของคุณ ต้องได้ `RESULT: 84 passed, 0 failed`
2. สร้างไฟล์ทดลองใหญ่ ๆ: `java MakeTestFile shared/test.bin 500`
3. เปิด server แล้วรัน `java Client localhost 5000 benchmark test.bin 3` เก็บตารางและ `benchmark_results.csv` ไว้ใส่รายงาน
4. ถ้ามีเวลา รัน 5 รอบ และลองปิดโปรแกรมอื่น เพื่อให้ผลนิ่งขึ้น
5. เขียนอธิบายผลด้วยตัวเลขที่ได้จริง (ห้ามคัดลอกตัวเลขจากตัวอย่างในเอกสาร) ตอบ 4 ประเด็น: (ก) 10 worker เร็วขึ้นกี่เท่า และทำไมไม่ถึง 10 เท่า (ข) nio ชนะหรือแพ้ io เพราะอะไร (ค) ข้อจำกัดของ localhost (ง) ผลตรวจ size/hash

5.2 ทำไม 10 worker ไม่เร็วขึ้น 10 เท่า และทำไม NIO ไม่ชนะเสมอ (สรุปสำหรับรายงาน)

ประเด็น	เหตุผล
คอขวดร่วม	ทุก worker ใช้ **ดิสก์เดียว** (หรือ page cache เดียว) และ **เครือข่าย/loopback เดียว** ถ้าคอขวดคือทรัพยากรร่วม เพิ่ม worker ก็ไม่ช่วย
localhost ไม่ใช่เครือข่ายจริง	ข้อมูลวิ่งในหน่วยความจำของ OS ไม่มี latency ของเครือข่ายให้ "ซ่อน" ด้วยการดาวน์โหลดขนาน และความต่างระหว่าง io/nio มักเล็กกว่าความแกว่งของแต่ละรอบ จึงต้องดูค่าเฉลี่ยหลายรอบ
Page cache	ไฟล์ที่เพิ่งอ่านอยู่ใน RAM แล้ว รอบถัดไปเร็วผิดปกติ (จึงมี warm-up และควรใช้ไฟล์ใหญ่กว่า RAM ถ้าอยากวัดดิสก์จริง)
ค่าใช้จ่ายของ worker	สร้าง thread + เปิด connection 10 เส้น + จัดการช่วงข้อมูล ถ้าไฟล์เล็ก ค่าใช้จ่ายนี้มากกว่าเวลาโอนข้อมูล
CPU และขั้นตอนที่เป็นลำดับ	worker มากกว่าแกน CPU จะสลับ thread แย่งกัน · การถามขนาดไฟล์ก่อนเริ่มและ join รอ worker ที่ช้าที่สุดยังเป็นลำดับ (กฎของ Amdahl)
NIO ฝั่งส่ง vs ฝั่งรับ	ฝั่งส่ง `transferTo` ใช้ `sendfile` ลดการคัดลอกและ system call แต่ฝั่งรับ `transferFrom` จาก socket ไม่ใช่ zero-copy แท้ (JDK อ่านทีละ ~8 KB ผ่าน buffer ชั่วคราว) จึงอาจไม่เร็วกว่า buffer 64 KB ของ io

ส่วนที่ 6 บั๊กที่พบ ข้อจำกัด และผลทดสอบ

คุณสงสัยว่าโค้ดยังมีบั๊ก จึงตรวจซ้ำโดยอ่านโค้ดทุกไฟล์ แล้วลองรันสถานการณ์ที่เสี่ยงจริง (ไม่ใช่แค่อ่าน) ผลสรุปอยู่ในส่วนนี้ บั๊กที่ยืนยันแล้วถูกแก้และเพิ่ม test ป้องกันไว้

6.1 บั๊กที่พบในรอบนี้และแก้แล้ว

#	ปัญหา	ผลกระทบ	วิธีแก้
1	CSV/ตารางจัดรูปตัวเลขตาม locale ของเครื่อง (`%.3f`)	เครื่องภาษาเยอรมัน/ฝรั่งเศสเขียน `0,006` จุลภาคทำให้คอลัมน์ CSV เลื่อน เปิดใน Excel ผิด	ใช้ `Locale.ROOT` ทุกที่ที่จัดรูปตัวเลข (`Benchmark`, `RangeWorker`, `Client`) + test `germanCsvTest`
2	ใส่จำนวน worker/รอบ/พอร์ตที่ไม่ใช่ตัวเลข เช่นลืม workers แล้วพิมพ์ `nio`	ข้อความ error คลุมเครือ (`For input string: "nio"`)	เมธอด `Client.parseNumber` บอกชัดว่าค่าไหนผิด + test `numberWordTest`
3	`java Server 99999` (พอร์ตนอกช่วง)	ได้ stack trace ยาว ๆ	ตรวจ 1–65535 ใน `Server.main` พิมพ์ข้อความสั้น exit 1

ปัญหาที่แก้ไปแล้วก่อนหน้านี้: เปรียบเทียบ offset+length ที่อาจล้น, ไฟล์ปลายทางเสียหลัง worker ล้ม (ตอนนี้ลบทิ้ง), `accept()` ล้มแล้ว server ตาย (ตอนนี้ลองใหม่), คำสั่งภาษาตุรกี, connection ที่เงียบค้าง thread (timeout 60 วินาที), พิมพ์โหมดผิดแล้วถูกตีเป็น io เงียบ ๆ, host ผิดแล้วได้ `Error: null`

6.2 สิ่งที่ลองรันจริงเพื่อยืนยัน

สถานการณ์	ผล
คอมไพล์ด้วย `javac --release 11`	ผ่าน (โค้ดไม่ใช้ฟีเจอร์ Java 12+)
`java TestRunner`	**84/84 ผ่าน**
ไฟล์ขนาด 2.5 GB (เกิน 2^31 byte) io และ nio	ได้ไฟล์ถูกต้อง SHA-256 ตรง (ยืนยันว่า `long` ไม่ล้น)
หลาย client พร้อมกัน	worker ทำงานซ้อนเวลากันจริง (ดูเวลาเริ่ม/จบของแต่ละ worker)
ไฟล์เปิดเต็มโควตา (Too many open files)	server ไม่ตาย กลับมารับต่อได้
ชื่อไฟล์ภาษาไทย (ตั้ง locale UTF-8)	ทำงานได้ · ถ้า locale เป็น ASCII/POSIX ชื่อจะเพี้ยน (ปัญหาของระบบปฏิบัติการ ไม่ใช่ของโค้ด)
ข้อผิดพลาดบน command line (พอร์ตผิด, host ผิด, ไม่มีไฟล์, workers=0 ฯลฯ)	ได้ข้อความสั้นอ่านรู้เรื่อง ไม่มี stack trace

> **สิ่งที่ยังไม่ได้ทดสอบ**
> ยังไม่ได้รันบน Windows / macOS, บนเครือข่ายจริง (ไม่ใช่ localhost) และบน JDK 11 จริง (ตรวจแค่คอมไพล์) ถ้ามีเครื่องอื่น แนะนำรัน `java TestRunner` ซ้ำ

ผลที่ถูกต้องของ `java TestRunner` คือบรรทัดสุดท้ายแบบนี้:

```
================================================
 RESULT: 84 passed, 0 failed
================================================
```

6.3 ข้อจำกัดที่ยังเหลือ (ตั้งใจไม่แก้ เพื่อให้โค้ดเรียบง่ายระดับปี 3)

ข้อจำกัด	รายละเอียด	ควรทำอย่างไรถ้าถูกถาม
Client ไม่มี timeout	ถ้า server เงียบ client จะค้างรอ (การใส่ timeout สั้นอาจตัดการรอ HASH ของไฟล์ใหญ่ที่ช้าโดยผิดพลาด)	ยอมรับว่าเป็นข้อจำกัด อธิบายเหตุผลข้างต้น
Server ไม่จำกัดจำนวน thread	client ที่ต่อมาก ๆ และส่ง GET แล้วไม่อ่าน อาจถือ thread ค้าง (วิเคราะห์ ยังไม่ได้ทดสอบ) ใช้ thread pool หรือ virtual thread (Java 21) แก้ได้	โจทย์อนุญาตให้ใช้ Thread / Thread Pool / Virtual Thread
symlink ใน `shared/`	ลิงก์ที่ชี้ออกนอกโฟลเดอร์ **ถูกส่งให้** (ยืนยันแล้ว) เพราะตรวจแค่ชื่อไฟล์ ไม่ได้ตรวจพาธจริง	ไม่ควรใส่ symlink ในโฟลเดอร์ที่แชร์ หรือเพิ่มการเทียบ canonical path
ชื่อไฟล์มีช่องว่าง / `:` / `..`	ไม่รองรับ (protocol แยกคำด้วยช่องว่าง) และ LIST จะไม่แสดงไฟล์เหล่านี้	ตั้งชื่อไฟล์แบบ `test.bin` หรือ `my_file.txt`
ฟังทุก interface, ไม่มี authentication	ใครในเครือข่ายเดียวกันก็โหลดไฟล์ในโฟลเดอร์ที่แชร์ได้	ใช้เฉพาะในแล็บ/เครื่องตัวเอง และแชร์เฉพาะโฟลเดอร์ทดลอง
HASH คำนวณใหม่ทุกครั้ง	ไฟล์ใหญ่ใช้เวลาหลายวินาที	ไม่ถูกนับในเวลาที่วัด
NIO ฝั่ง client ไม่ใช่ zero-copy แท้	เป็นข้อจำกัดของ JDK เมื่อต้นทางเป็น socket	อธิบายในรายงาน (ส่วน 5.2)
ไม่มี retry / resume	worker ล้ม = ทั้งการดาวน์โหลดล้ม และลบไฟล์ปลายทาง	นอกขอบเขตโจทย์
CSV เหลือหัวตารางอย่างเดียวถ้า benchmark ล้มก่อนมีผล	ไฟล์ `benchmark_results.csv` มีแต่บรรทัดหัว	รันใหม่หลังแก้ปัญหา
`MakeTestFile` ไม่ตรวจว่าขนาดเป็นตัวเลข	พิมพ์ผิดได้ stack trace	เป็นเครื่องมือช่วย ไม่ใช่ส่วนของระบบ

ส่วนที่ 7 คำถามที่อาจถูกถามตอนนำเสนอ

ถาม: ทำไมแต่ละ worker ต้องมี connection ของตัวเอง
ตอบ: เพราะโจทย์ต้องการดาวน์โหลดขนาน ถ้าใช้ connection เดียว ข้อมูลของทุกช่วงต้องเรียงต่อกันใน stream เดียว ไม่ขนานจริง และ protocol นี้ตอบ 1 คำสั่งต่อ 1 connection ทีละคำสั่ง

ถาม: ทำไมไม่ต้อง lock ตอนหลาย thread เขียนไฟล์เดียวกัน
ตอบ: ทุก worker เขียนคนละช่วงไม่ทับกัน และแต่ละตัวเปิด `RandomAccessFile` ของตัวเอง ไม่มีตัวแปรหรือตำแหน่งไฟล์ร่วม จึงไม่มีอะไรให้แย่งกัน ส่วนฟิลด์ `error` ที่ main อ่าน ปลอดภัยเพราะอ่านหลัง `join()`

ถาม: worker ตัวสุดท้ายรับเศษอย่างไร
ตอบ: `length = size - offset` สำหรับตัวสุดท้าย เช่น ไฟล์ 1,000,003 byte: 9 ตัวแรกได้ 100,000 ตัวสุดท้ายได้ 100,003

ถาม: ทำไมต้อง `setLength(size)` ก่อน
ตอบ: ให้ไฟล์มีขนาดเต็มก่อน เพื่อให้ทุก worker เขียนที่ offset ของตนได้ และเพราะ `transferFrom` จะไม่ถ่ายข้อมูลถ้าตำแหน่งเกินขนาดไฟล์

ถาม: ทำไม server ต้องส่ง `OK <length>` ก่อนข้อมูล
ตอบ: ให้ client รู้ว่าจะต้องอ่านกี่ byte และตรวจว่าตรงกับที่ขอ เพราะข้อมูลไฟล์เป็น byte ดิบ ไม่มีตัวปิดท้าย

ถาม: ทำไมอ่าน header ทีละ byte
ตอบ: `BufferedReader` อ่านล่วงหน้าเป็นก้อน อาจกลืน byte ของไฟล์ที่ตามหลัง header ทำให้ไฟล์เสีย

ถาม: Traditional I/O กับ NIO transferTo ต่างกันอย่างไร
ตอบ: io: ไฟล์ → byte[] ในโปรแกรม → socket (คัดลอกผ่านโปรแกรม) · nio: ให้ OS ส่งจากไฟล์ไป socket เอง (`sendfile`) ไม่ผ่าน `byte[]` จึงลดการคัดลอกและการสลับ user/kernel

ถาม: NIO ฝั่ง client เป็น zero-copy ไหม
ตอบ: ไม่แท้ เมื่อต้นทางเป็น socket JDK อ่านทีละ ~8 KB เข้า buffer ชั่วคราวแล้วเขียนลงไฟล์ (ฝั่ง server ที่ใช้ `transferTo` เป็น zero-copy จริง)

ถาม: ทำไม 10 workers ไม่เร็วขึ้น 10 เท่า
ตอบ: คอขวดร่วม (ดิสก์/page cache/loopback), ค่าใช้จ่ายสร้าง thread และ connection, จำนวนแกน CPU, ขั้นตอนที่ยังเป็นลำดับ (ดูตารางส่วน 5.2)

ถาม: ทำไมต้อง warm-up
ตอบ: รอบแรกช้ากว่าเพราะ JVM โหลด/คอมไพล์โค้ดและไฟล์ยังไม่อยู่ใน cache ของ OS ถ้านับรวม จะทำให้กรณีที่รันก่อนดูช้าเกินจริง

ถาม: ทำไม hash ไม่นับในเวลา
ตอบ: โจทย์ให้วัดเวลาดาวน์โหลด การคำนวณ SHA-256 เป็นขั้นตรวจสอบ ไม่ใช่การโอนข้อมูล

ถาม: ป้องกัน client ขอไฟล์นอกโฟลเดอร์ได้อย่างไร
ตอบ: `isAllowedName` ปฏิเสธ `/ \ :` และ `..` (ข้อจำกัด: symlink ในโฟลเดอร์ที่แชร์ยังตามไปได้)

ถาม: ถ้า worker ตัวหนึ่งล้มจะเกิดอะไร

ตอบ: เก็บข้อความไว้ใน `error` main ตรวจหลัง `join()` แล้วลบไฟล์ปลายทางและโยน exception → Client พิมพ์ error และ exit code 1

ถาม: ถ้าต้องใช้ thread pool หรือ virtual thread ต้องแก้ตรงไหน

ตอบ: แก้เฉพาะ `Server.main` : แทน `new Thread(handler).start()` ด้วย `executor.submit(handler)` (`ClientHandler` เป็น `Runnable` อยู่แล้ว) ใน Java 21 ใช้ `Executors.newVirtualThreadPerTaskExecutor()`

ภาคผนวก คำสั่ง รหัส และศัพท์

A. คำสั่งทั้งหมด

```
# compile (ทำใหม่ทุกครั้งที่แก้โค้ด)
javac *.java

# test อัตโนมัติ 84 ข้อ
java TestRunner

# สร้างไฟล์ทดลอง 500 MB
java MakeTestFile shared/test.bin 500

# เทอร์มินัลที่ 1: server
java Server 5000 shared

# เทอร์มินัลที่ 2: client
java Client localhost 5000 list
java Client localhost 5000 info test.bin
java Client localhost 5000 download test.bin out.bin            (10 workers, io)
java Client localhost 5000 download test.bin out.bin 10 nio     (10 workers, nio)
java Client localhost 5000 download test.bin out.bin 1 io       (1 worker, io)
java Client localhost 5000 benchmark test.bin 3
```

B. Protocol

คำสั่ง	ตอบสำเร็จ	ตอบผิดพลาด
`LIST`	`FILE <name> <size>` (หลายบรรทัด) แล้ว `END`	—
`INFO <file>`	`SIZE <bytes>`	400, 404
`HASH <file>`	`SHA256 <hex 64 ตัว>`	400, 404
`GET <file> <offset> <length> [io|nio]`	`OK <length>` + byte ดิบ	400, 404, 416
คำสั่งอื่น	—	`ERROR 400 Unknown command`

C. ข้อความ error ที่พบบ่อยและความหมาย

ข้อความ	สาเหตุ	วิธีแก้
`Connection refused`	server ยังไม่เปิด หรือพอร์ตไม่ตรง	เปิด `java Server` ก่อน และเช็คเลขพอร์ต
`Cannot open port 5000 ...`	พอร์ตถูกใช้อยู่ (เปิด server ซ้ำ)	ปิดตัวเก่า หรือใช้พอร์ตอื่น
`Cannot find host ...`	ชื่อ host ผิด	ใช้ `localhost` หรือ IP
`Server replied: ERROR 404 File not found`	ชื่อไฟล์ไม่ตรง/มีอักขระต้องห้าม	ใช้ `list` ดูชื่อที่ถูกต้อง

`workers must be between 1 and 100`	จำนวน worker นอกช่วง	ใส่ 1–100
`A worker failed: ...`	worker ตัวใดตัวหนึ่งล้ม	ดูข้อความต่อท้าย (เช่น server ปิดกลางทาง)
`DAMAGED` / exit code 2	ขนาดหรือ SHA-256 ไม่ตรง	อย่าใช้ตัวเลขที่ได้ ตรวจว่าไฟล์ต้นฉบับไม่ถูกแก้ระหว่างโหลด
`Could not find or load main class`	ยังไม่ได้ compile หรืออยู่ผิดโฟลเดอร์	`javac *.java` ในโฟลเดอร์ที่มีไฟล์ .java

D. ศัพท์

ศัพท์	ความหมาย
Socket / SocketChannel	ปลายทางของ connection เครือข่าย (SocketChannel คือแบบ NIO)
Thread	สายการทำงานหนึ่งสายในโปรแกรม ทำงานคู่ขนานกับสายอื่น
start() / join()	สั่งให้ thread เริ่ม / รอให้ thread นั้นจบ
offset / length	ตำแหน่งเริ่ม (byte ที่เท่าไร) / จำนวน byte ที่ต้องการ
Byte range	ช่วง [offset, offset + length) ของไฟล์
Positional write	เขียนที่ตำแหน่งที่ระบุ (เช่น `seek` แล้ว `write`)
Throughput	ปริมาณข้อมูลต่อวินาที (MB/s) = ขนาดไฟล์ ÷ เวลา
Zero-copy	ให้ OS ย้ายข้อมูลจากไฟล์ไป socket โดยไม่คัดลอกผ่านหน่วยความจำของโปรแกรม
sendfile	system call ของ Linux ที่ `transferTo` เรียกใช้
Page cache	ส่วนของ RAM ที่ OS ใช้เก็บข้อมูลไฟล์ที่เพิ่งอ่าน ทำให้อ่านซ้ำเร็ว
Loopback / localhost	เครือข่ายเสมือนในเครื่องเดียว ไม่มีสายจริง
SHA-256	ฟังก์ชันสร้างลายนิ้วมือ 256 บิต (64 ตัวอักษรฐาน 16) ของข้อมูล
Race condition	ผลผิดเพราะหลาย thread แย่งแก้ข้อมูลเดียวกัน (โปรแกรมนี้เลี่ยงโดยแบ่งช่วงไม่ทับกัน)
Path traversal	การใช้ `../` เพื่ออ่านไฟล์นอกโฟลเดอร์ที่แชร์
Overflow	ตัวเลขเกินค่าสูงสุดของชนิดข้อมูลแล้ววนเป็นค่าติดลบ
Warm-up	รันก่อนหนึ่งรอบโดยไม่นับ เพื่อให้สภาพเครื่อง/JVM นิ่ง